

BAB IV

HASIL DAN PAMBAHASAN

4.1. Hasil Penelitian

Bagian ini menyajikan hasil dari proses penelitian, mencakup rincian tahapan pengembangan sistem, data hasil uji coba sistem, serta prototipe sistem akhir yang dihasilkan.

4.1.1. Membuat Sistem

4.1.1.1. Fase Analisa Kebutuhan

4.1.1.1.1. Analisa Kebutuhan Fungsional

1. Sistem harus mampu melakukan kontrol *pH* dan konsentrasi nutrisi secara otomatis.
2. Parameter ambang batas untuk kontrol otomatis harus dapat diatur dan disimpan oleh pengguna melalui *mobile app*.
3. Pengguna harus dapat memantau data *real-time pH* dan konsentrasi nutrisi melalui *mobile app*.
4. Pengguna harus dapat melakukan kontrol dosis *pH* dan nutrisi secara manual melalui *mobile app*.
5. Sistem harus dapat menampilkan data *real-time pH* dan konsentrasi nutrisi langsung pada antarmuka perangkat *IoT*.
6. Pengguna harus dapat mengoperasikan kontrol manual dosis *pH* dan nutrisi langsung melalui antarmuka perangkat *IoT*.
7. Sistem harus dapat mengirim notifikasi ke *mobile app* ketika volume larutan koreksi *pH* atau nutrisi berada di bawah batas minimum.
8. Sistem harus dapat mengirim notifikasi ke *mobile app* ketika perangkat *IoT* terputus (*offline*) dari server.
9. Perangkat *IoT* harus dapat memberikan peringatan ketika volume larutan koreksi habis.

4.1.1.1.2. Analisa Kebutuhan Non Fungsional

a. Kinerja

Kinerja sistem difokuskan pada kecepatan dan efisiensi dalam melayani pengguna. Kebutuhan utama adalah kecepatan respons yang cepat, memastikan

bahwa komunikasi data antara perangkat *IoT*, server, dan *mobile app* memiliki latensi rendah. Hal ini menjamin bahwa setiap perintah kontrol yang dikirim pengguna dan setiap data *real-time* yang diminta ditampilkan dalam waktu yang cepat dan dapat diterima, sehingga pengalaman pengguna tidak terganggu oleh keterlambatan sistem.

b. Skalabilitas

Kebutuhan skalabilitas difokuskan pada kemampuan sistem untuk beradaptasi terhadap pertumbuhan di masa depan. Hal ini diwujudkan melalui arsitektur yang fleksibel dan terdistribusi, yang memungkinkan penambahan volume data, jumlah pengguna, atau unit perangkat *IoT* baru tanpa harus melakukan perancangan ulang sistem secara keseluruhan. Arsitektur yang dirancang harus memudahkan *upgrade* dan ekspansi, menjamin sistem tetap dapat mengakomodasi pertumbuhan kebutuhan fungsional dan teknis di masa mendatang.

c. Keamanan

Aspek keamanan difokuskan pada perlindungan akses sistem dan kerahasiaan kode. Untuk melindungi koneksi data, setiap komunikasi MQTT harus melakukan autentikasi menggunakan kredensial *username* dan *password* unik yang terdaftar pada broker. Selain itu, untuk mencegah upaya *reverse engineering* dan melindungi kekayaan intelektual kode aplikasi, *mobile app* akan menerapkan teknik *obfuscation* pada kode *build* menggunakan ProGuard/R8.

d. Keandalan

Keandalan sistem diukur dari kemampuannya untuk beroperasi secara terus-menerus dan pulih dari kegagalan. Sistem harus memiliki kemampuan pemulihan kegagalan, di mana jika terjadi *restart* atau pemadaman listrik pada perangkat *IoT*, perangkat dapat menghubungkan ulang ke server secara otomatis tanpa intervensi manual. Kemudian, sistem harus memiliki toleransi jaringan, yang memungkinkan perangkat untuk menyimpan data sementara saat koneksi terputus dan mengirimkannya ketika jaringan kembali normal. Selain itu, untuk memastikan keberlangsungan operasional di lingkungan lembap, perangkat *IoT* harus memiliki ketahanan fisik terhadap percikan air, yang diwujudkan melalui penggunaan penutup dengan standar proteksi yang memadai.

e. Kemudahan Penggunaan

Sistem harus mudah digunakan oleh pengguna. Hal ini dapat diwujudkan melalui antarmuka yang intuitif pada *mobile app* yang memiliki alur navigasi yang jelas, sehingga meminimalkan waktu yang dibutuhkan pengguna untuk mempelajarinya. Selain itu, sistem perlu menjamin kompatibilitas perangkat yang optimal, memastikan bahwa aplikasi dapat berjalan dengan baik pada mayoritas versi sistem operasi android yang digunakan pengguna.

f. Kemudahan Pemeliharaan

Kemudahan pemeliharaan menjadi kebutuhan penting untuk memfasilitasi pengembangan dan perbaikan di masa depan. Hal ini dicapai melalui penggunaan struktur kode modular yang rapi, baik pada *firmware IoT* maupun kode *mobile app*. Struktur ini memastikan bahwa perbaikan, penambahan fitur, atau modifikasi pada satu bagian sistem tidak akan mengganggu fungsionalitas bagian lainnya.

g. Kompatibilitas

Kompatibilitas sistem ditingkatkan melalui pemanfaatan *n8n* sebagai *workflow automation tool*. Penggunaan *tool* ini memberikan kemudahan integrasi sistem dengan berbagai layanan eksternal pihak ketiga (seperti Telegram, Google Sheets, atau layanan *cloud* lainnya) di masa depan. Hal ini mengurangi ketergantungan pada penulisan kode secara manual untuk integrasi layanan baru.

h. Aksesibilitas

Aksesibilitas ditujukan untuk memastikan bahwa sistem dapat digunakan secara efektif oleh berbagai kalangan pengguna. Aspek utama adalah kejelasan informasi visual, di mana tampilan data dan teks pada *mobile app* maupun perangkat *IoT* harus menggunakan ukuran teks yang memadai dan kontras warna yang tinggi. Hal ini diperlukan untuk memudahkan pembacaan oleh pengguna, terutama bagi mereka yang memiliki keterbatasan penglihatan.

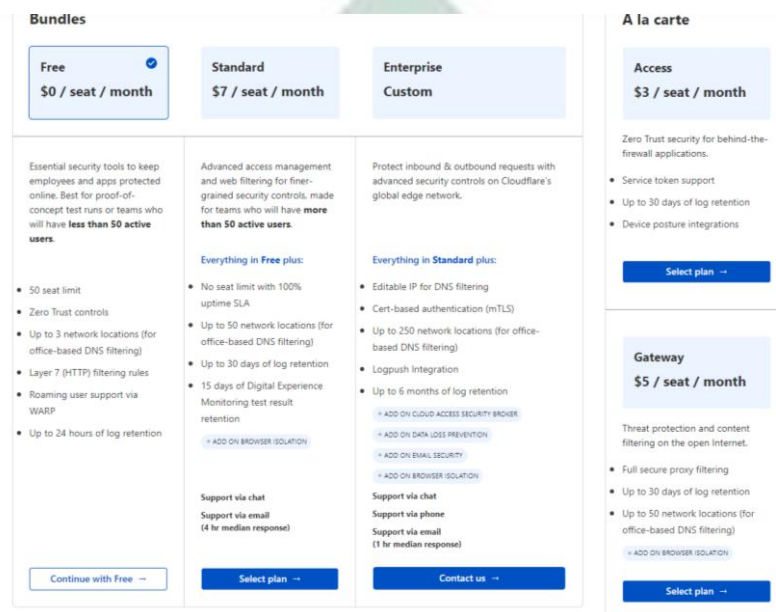
4.1.1.1.3. Analisa Batasan Layanan Pihak Ketiga

Selain kebutuhan fungsional dan non-fungsional sistem, penelitian ini juga mengidentifikasi batasan operasional yang timbul dari penggunaan infrastruktur pihak ketiga pada skema tanpa biaya (*free tier*). Analisa ini bertujuan untuk

memvalidasi bahwa batasan kuota dan spesifikasi layanan gratis tersebut tetap dapat mengakomodasi kebutuhan lalu lintas data sistem prototipe secara stabil.

a. *Cloudflare Tunnel (Zero Trust Free Plan)*

Layanan ini diimplementasikan sebagai gerbang keamanan untuk mengekspos layanan n8n dan MQTT Broker agar dapat diakses oleh *mobile app* dari jaringan publik. Perlu dicatat bahwa komunikasi data sensor dari perangkat *IoT* (ESP32) dilakukan sepenuhnya melalui jaringan lokal (*Local Area Network*), sehingga tidak membebani lalu lintas data pada *tunnel*.



Gambar 4.1. Paket Cloudflare

Berdasarkan spesifikasi paket *Free Plan* yang diadopsi (Gambar 4.1), terdapat batasan akses hingga maksimal 50 pengguna aktif (*seats*) dan retensi log aktivitas jaringan selama 24 jam. Analisa kebutuhan menunjukkan bahwa batasan ini sangat aman, mengingat *tunnel* hanya digunakan untuk jalur komunikasi kontrol dari *mobile app* oleh pengguna tunggal, bukan untuk transmisi data sensor yang intensif.

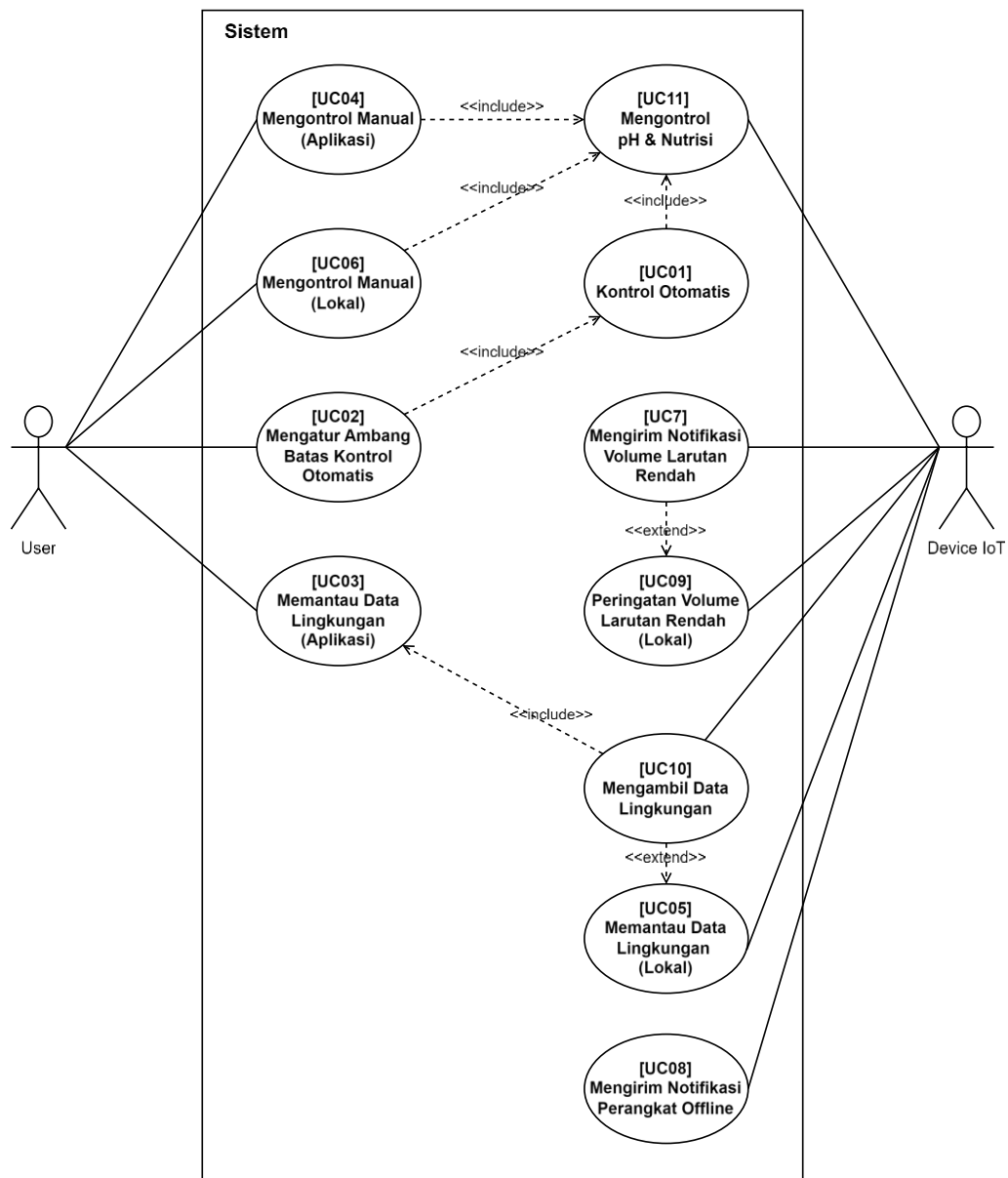
b. *Firebase Cloud Messaging (FCM)*

FCM digunakan sebagai layanan perantara untuk mengirimkan notifikasi peringatan (*alert*) dari server n8n ke aplikasi Android secara *real-time*. Mengacu pada dokumentasi teknis Google terkait *Throttling and Quotas (FCM Throttling*

and Quotas, n.d.), layanan ini menerapkan batasan ukuran muatan (*payload*) maksimum sebesar 4 KB (4096 bytes) per pesan. Batasan ini tidak menjadi kendala operasional, mengingat format data yang dikirimkan sistem hanya berupa *string* teks pendek berisi informasi status sensor tanpa lampiran media (gambar/video), sehingga ukurannya dipastikan berada dalam rentang aman layanan gratis yang disediakan.

4.1.1.2. Fase Desain Sistem

4.1.1.2.1. Uce Case Diagram



Gambar 4.2. Use Case Diagram

Gambar 4.2 merupakan *use case diagram* sistem, menggambarkan interaksi antara dua aktor utama, yaitu *User* dan *Device IoT*, dengan Sistem secara keseluruhan. *User* memiliki peran penting dalam mengendalikan dan memantau sistem. Secara spesifik, *User* dapat melakukan kontrol manual terhadap sistem baik melalui aplikasi (UC04) maupun secara lokal dari perangkat itu sendiri (UC06). Kedua *use case* kontrol manual (UC04 dan UC06) meng-include UC11 (mengontrol *pH* dan nutrisi). Selain itu, UC01 (kontrol otomatis) juga meng-include UC11, yang berarti setiap fungsi kontrol, baik manual maupun otomatis, mencakup tindakan mengontrol *pH* dan nutrisi. *User* juga bertanggung jawab untuk mengatur batasan operasional melalui UC02 untuk mengatur ambang batas kontrol otomatis, yang sangat penting sebagai prasyarat bagi fungsi UC01.

Kontrol dan pemantauan lokal pada sistem diwujudkan melalui interaksi fisik dengan perangkat. Mengontrol manual secara lokal (UC06) memungkinkan *User* untuk langsung berinteraksi menggunakan tombol fisik yang tersedia pada perangkat. Sementara itu, memantau data lingkungan secara lokal (UC05) memberikan umpan balik langsung kepada *User*, termasuk melalui indikator LED dan layar LCD lokal. Fungsi kontrol dan pemantauan lokal ini sangat penting untuk pengoperasian sistem saat koneksi jaringan tidak tersedia (*offline*). Detail implementasi untuk kontrol lokal, termasuk fungsi setiap tombol dan arti setiap indikator LED, disajikan pada Tabel 4.1 dan Tabel 4.2 berikut:

Tabel 4.1. Detail Indikator LED

Nama Indikator	Warna LED	Keterangan
Indikator Power	Hijau	Menyala terus-menerus saat perangkat mendapatkan catu daya.
<i>Heartbeat</i>	Merah	Berkedip secara teratur untuk menandakan bahwa program utama pada perangkat berjalan dengan normal.
<i>Warning</i>	Merah	Berkedip saat ada <i>warning</i> , seperti ketika tangki koreksi atau tangki utama berdada pada batas minimum.
Status WiFi	Biru	Berkedip saat sedang mencari atau mencoba terhubung ke jaringan WiFi. Menyala terus-menerus saat berhasil terhubung.
Status MQTT	Biru	Menyala terus-menerus saat berhasil terhubung ke <i>MQTT Broker</i> . Berkedip setiap

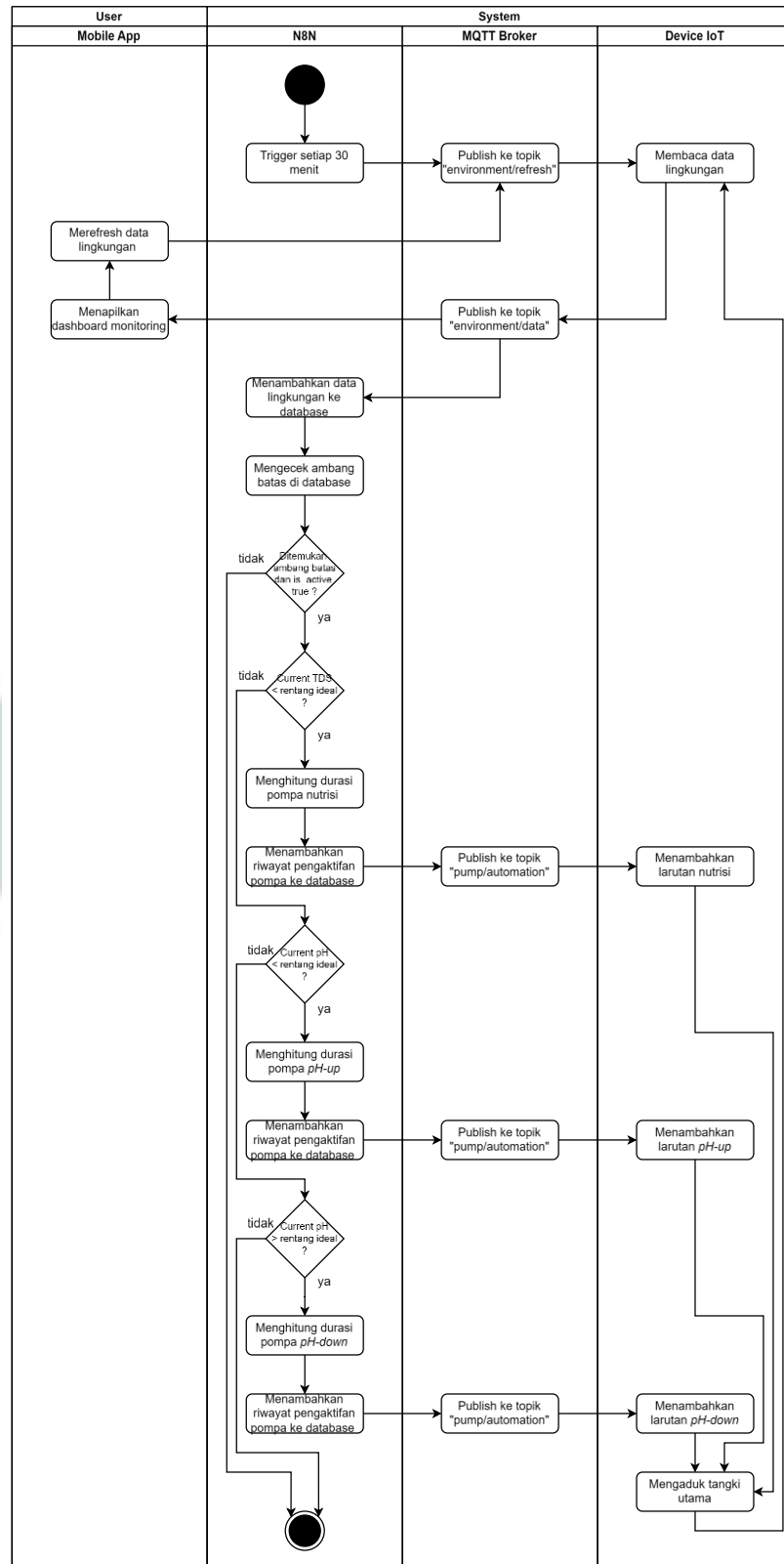
Nama Indikator	Warna LED	Keterangan
		kali ada proses pengiriman atau penerimaan data.
Pembacaan Data	Kuning	Menyala ketika perangkat sedang melakukan pembacaan data lingkungan.
Pemberian Larutan	Kuning	Menyala ketika perangkat sedang menambahkan larutan koreksi.

Tabel 4.2. Detail Tombol Aksi

Nama Tombol	Warna Tombol	Keterangan
<i>Reconnect</i>	Biru	Sistem akan mencoba menghubungkan ulang ke WiFi dan <i>MQTT Broker</i> .
Baca Data	Hijau	Sistem akan melakukan pembacaan data lingkungan.
Tambah <i>pH-Up</i>	Kuning	Sistem akan menambahkan larutan <i>pH-Up</i> ke tangki utama.
Tambah <i>pH-Down</i>	Kuning	Sistem akan menambahkan larutan <i>pH-Down</i> ke tangki utama.
Tambah Nutrisi	Kuning	Sistem akan menambahkan larutan nutrisi ke tangki utama.

Aktor *Device IoT*, sebagai aktor kedua, memiliki tanggung jawab untuk menjalankan perintah dan memantau lingkungan secara mandiri. UC05 ini secara otomatis meng-include UC10 (mengambil data lingkungan), menunjukkan bahwa pengambilan data adalah bagian integral dari pemantauan lokal. Selanjutnya, *Device IoT* adalah aktor yang menjalankan *use case* kendali fisik, yaitu mengontrol *pH* dan nutrisi (UC11), dan secara otomatis menjalankan kontrol otomatis (UC01) berdasarkan ambang batas yang telah diatur oleh *User*. Terakhir, *Device IoT* terlibat dalam mekanisme notifikasi. Ketika UC07 (mengirim notifikasi volume larutan rendah) terjadi, kondisi ini dapat memperluas fungsi menjadi UC09 (peringatan volume larutan rendah secara lokal), yang memberikan peringatan langsung pada perangkat. *Device IoT* juga bertanggung jawab untuk mengirim notifikasi perangkat *offline* (UC08) kepada *User*.

a. AC01 Kontrol Otomatis



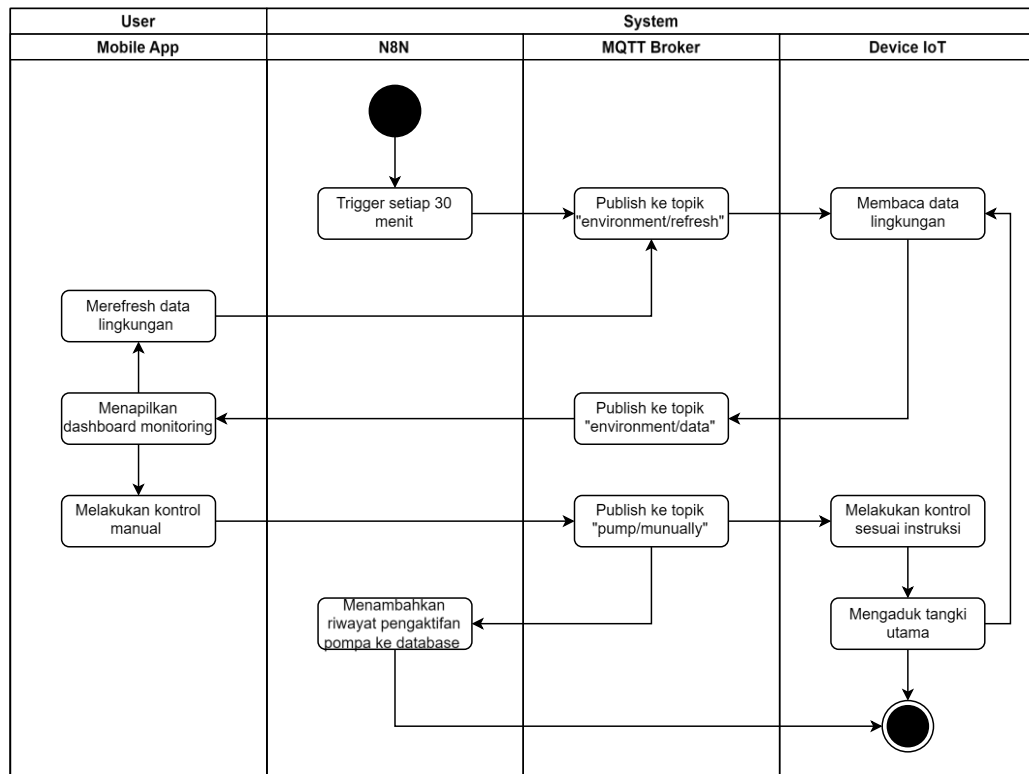
Gambar 4.3. AC01 Kontrol Otomatis

Berdasarkan Gambar 4.3, merupakan *activity diagram* memvisualisasikan alur otomatisasi sistem yang mengintegrasikan fungsi dari beberapa *use case*, yaitu UC01, UC11, UC03, dan UC10, terkait dengan pemantauan dan pemrosesan data lingkungan. Alur otomatisasi sistem diawali oleh N8N yang secara berkala setiap 30 menit memublikasikan perintah ke topik “*environment/refresh*”. Perintah ini memicu *Device IoT* untuk membaca data lingkungan dan mengirimkannya kembali ke topik “*environment/data*”. Selanjutnya, N8N menerima data tersebut, menyimpannya ke dalam *database*, kemudian memeriksa apakah terdapat konfigurasi ambang batas yang aktif. Jika tidak ada, alur otomatisasi untuk siklus ini akan berhenti.

Apabila ditemukan ambang batas yang aktif, N8N akan memulai proses perbandingan parameter air, diawali dengan pengecekan nilai TDS. Jika nilai TDS saat ini lebih rendah dari rentang ideal, N8N akan menghitung durasi pompa nutrisi, mencatat riwayat tindakan, dan memublikasikan perintah pengaktifan pompa ke topik “*pump/automation*”. *Device IoT* yang menerima perintah ini akan menambahkan larutan nutrisi dan mengaduk tangki. Namun, jika nilai TDS sudah sesuai, sistem akan melanjutkan ke pengecekan parameter *pH*.

Selanjutnya, sistem memeriksa nilai *pH*. Jika *pH* saat ini lebih kecil dari rentang ideal, proses koreksi yang sama seperti sebelumnya akan berjalan, namun kali ini untuk pompa *pH-Up*. Sebaliknya, jika *pH* saat ini lebih besar dari rentang ideal, maka pompa *pH-Down* yang akan diaktifkan. Setiap kali tindakan koreksi berhasil dijalankan oleh *Device IoT*, aktivitas akan kembali ke siklus pembacaan data lingkungan. Setelah semua pengecekan kondisi selesai, alur otomatisasi pada periode ini berakhir.

b. AC02 Kontrol Manual via Aplikasi



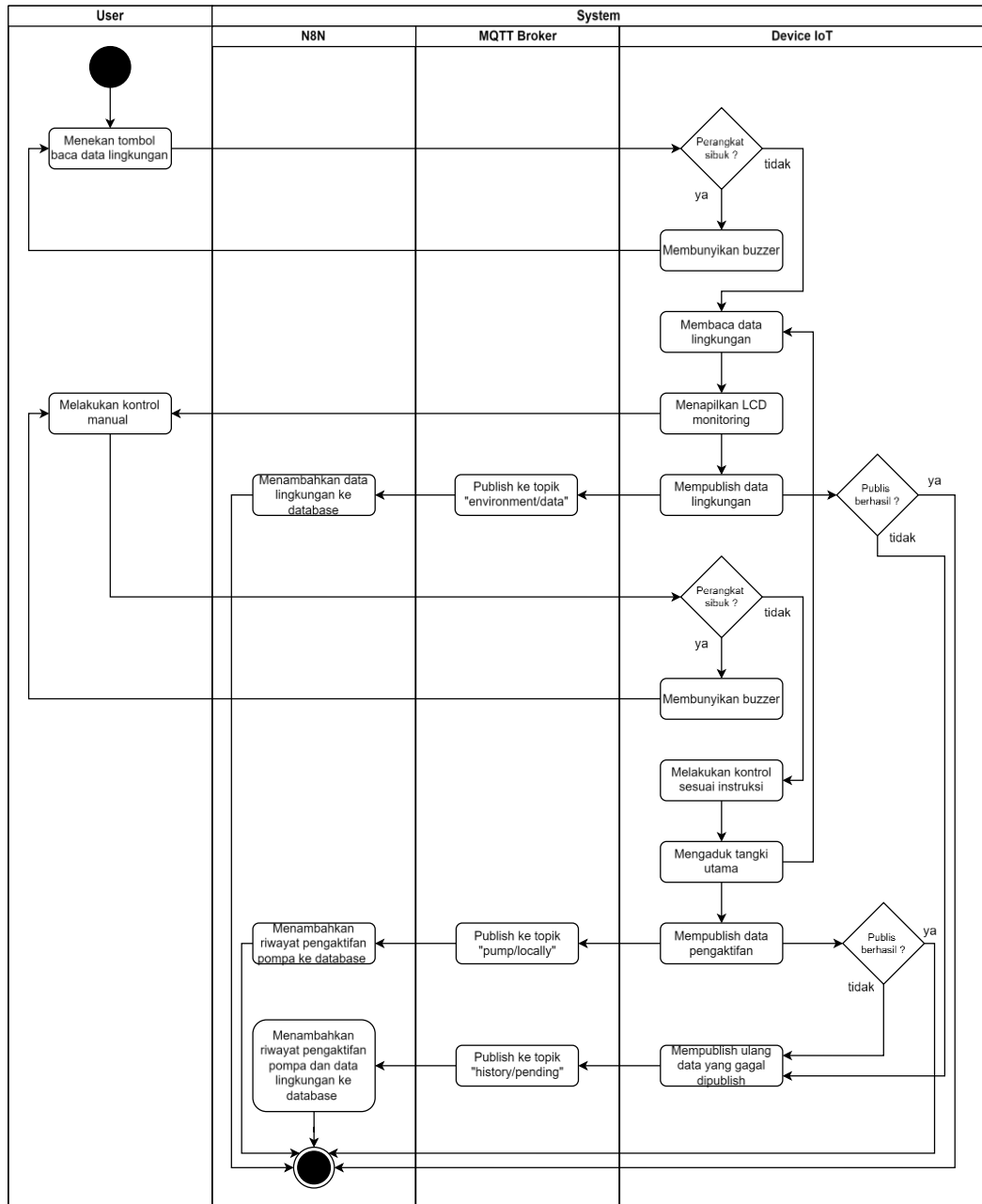
Gambar 4.4. AC02 Kontrol Manual via Aplikasi

Pada Gambar 4.4, merupakan *activity diagram* yang mengintegrasikan fungsi dari beberapa *use case*, yaitu UC04, UC11, UC03, dan UC10, terkait dengan pemantauan dan kontrol melalui aplikasi. Alur aktivitas kontrol manual diawali oleh N8N yang secara otomatis melakukan *trigger* setiap 30 menit. Proses ini dimulai dengan N8N yang memublikasikan perintah ke topik MQTT “*environment/refresh*”, yang kemudian diterima oleh *Device IoT* untuk memulai pembacaan data lingkungan. Setelah data terbaca, perangkat akan memublikasikannya kembali melalui *MQTT Broker* ke topik “*environment/data*”, sehingga nilainya dapat tampil pada *dashboard monitoring*. Pengguna juga memiliki opsi untuk memicu pengecekan ulang secara manual dengan mengirim perintah ke topik “*environment/refresh*” yang sama.

Selain memantau dan melakukan pengecekan ulang, pengguna dapat melakukan intervensi manual dengan memublikasikan perintah langsung ke topik “*pump/manually*”. Perintah ini memiliki dua tujuan: pertama, diterima dan disimpan ke dalam *database* oleh N8N untuk pencatatan; kedua, dieksekusi oleh

Device IoT untuk melakukan kontrol sesuai instruksi. Sebagai langkah terakhir setelah kontrol berhasil dijalankan, *Device IoT* akan mengaduk tangki utama, dan alur aktivitas kemudian kembali ke siklus pembacaan data atau selesai.

c. AC03 Kontrol Manual via Lokal



Gambar 4.5. AC03 Kontrol Manual via Lokal

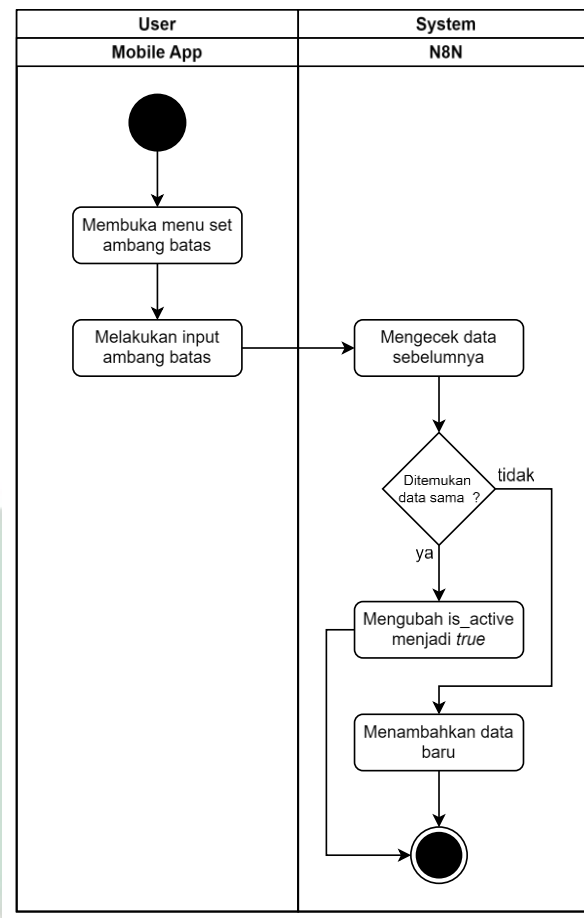
Berdasarkan Gambar 4.5, merupakan *activity diagram* yang mengintegrasikan fungsi dari beberapa *use case*, yaitu UC06, UC11, UC10, dan UC05, terkait dengan pemantauan dan kontrol langsung dari perangkat. Alur

dimulai ketika pengguna menekan tombol untuk membaca data lingkungan, di mana sistem akan terlebih dahulu memvalidasi status perangkat. Jika perangkat sedang sibuk, *buzzer* akan berbunyi sebagai notifikasi agar pengguna mencoba kembali nanti; namun jika perangkat dalam keadaan siap, sensor akan membaca data lingkungan dan menampilkannya pada *LCD monitoring*. Segera setelah data ditampilkan, sistem secara otomatis mencoba mengirimkan data tersebut ke topik "*environment/data*". Apabila pengiriman ini berhasil, N8N akan langsung menyimpan data lingkungan tersebut ke dalam database utama.

Setelah informasi ditampilkan pada LCD, pengguna dapat melanjutkan ke tahap kontrol manual dengan memberikan instruksi fisik ke perangkat. Sama seperti proses sebelumnya, perangkat akan melakukan validasi status kesibukan terlebih dahulu sebelum mengeksekusi perintah. Jika perangkat tidak sibuk, instruksi kontrol dijalankan, diikuti dengan proses pengadukan tangki utama untuk meratakan larutan. Setelah aksi mekanis selesai, perangkat akan memublikasikan data riwayat pengaktifan tersebut ke topik "*pump/locally*". Jika publikasi ini berhasil, N8N akan mencatat riwayat pengaktifan pompa tersebut ke dalam *database* sebagai log aktivitas yang sukses.

Untuk menjamin integritas data saat terjadi gangguan koneksi, diagram ini menerapkan mekanisme penanganan kegagalan pada setiap proses publikasi. Apabila terjadi kegagalan saat memublikasikan data lingkungan di tahap awal ataupun saat memublikasikan data pengaktifan pompa di tahap akhir, alur akan diarahkan ke proses publikasi ulang. Data yang gagal terkirim tersebut akan dialihkan ke topik "*history/pending*". N8N kemudian akan menangkap pesan dari topik ini dan menyimpannya menggunakan skema khusus yang menggabungkan riwayat pengaktifan pompa dan data lingkungan ke dalam *database*, memastikan tidak ada informasi yang hilang meskipun terjadi kendala jaringan pada saat kejadian.

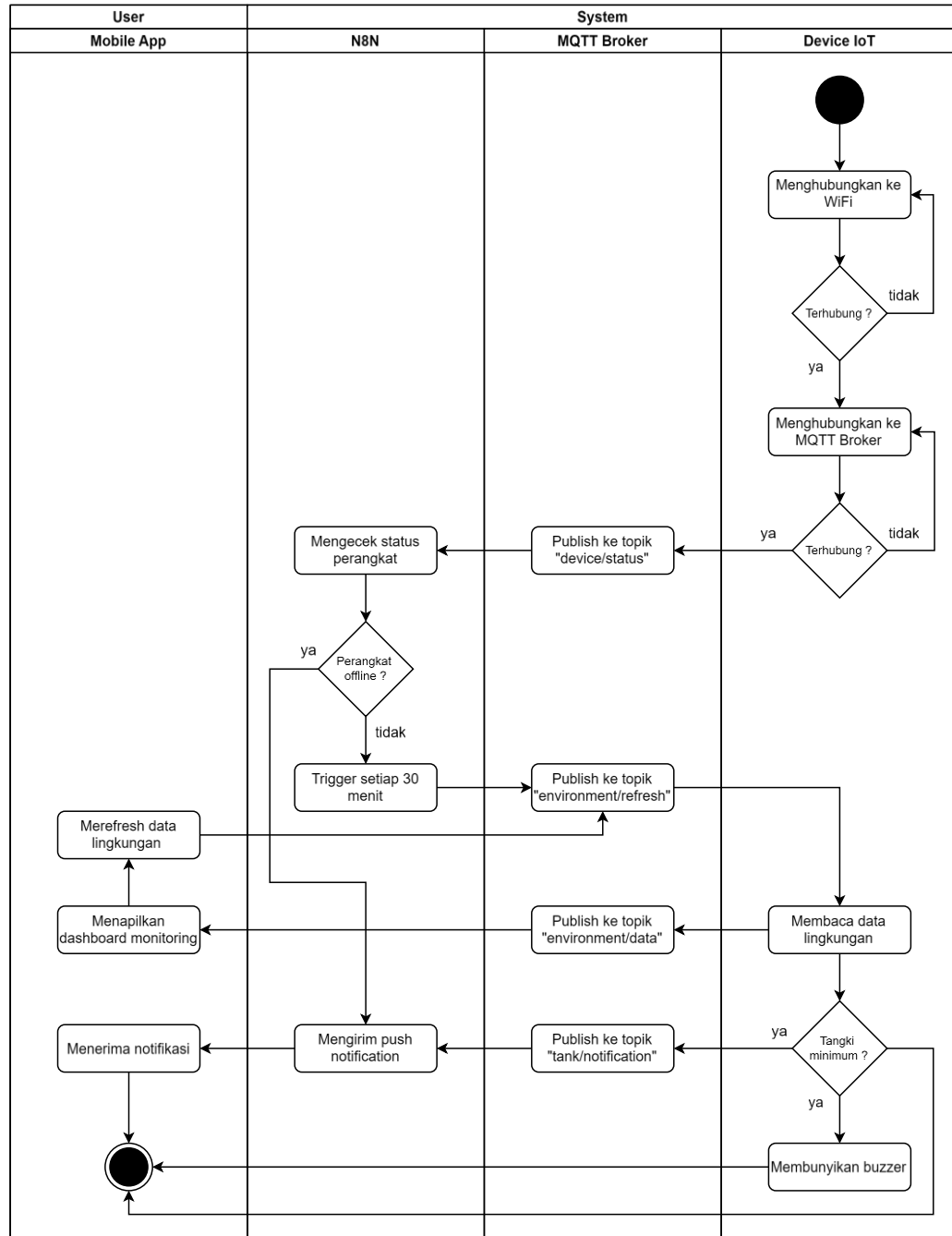
d. AC04 Mengatur Ambang Batas



Gambar 4.6. AC04 Mengatur Ambang Batas

Gambar 4.6 merupakan *activity diagram* memvisualisasikan alur *use case* UC02, mengatur ambang batas kontrol otomatis. Proses pengaturan dimulai ketika pengguna membuka menu untuk mengatur ambang batas, kemudian memasukkan rentang nilai ideal untuk *pH* dan kepekatan nutrisi. Setelah data tersebut diterima, N8N akan melakukan pengecekan ke dalam *database* untuk mencegah adanya duplikasi. Apabila ditemukan data dengan rentang nilai yang identik, sistem tidak akan membuat data baru, melainkan hanya mengubah status data yang sudah ada tersebut menjadi aktif. Namun, jika tidak ditemukan data yang serupa, maka sistem akan menyimpan *input* dari pengguna sebagai data ambang batas yang baru.

e. AC05 Sistem Notifikasi



Gambar 4.7. AC05 Sistem Notifikasi

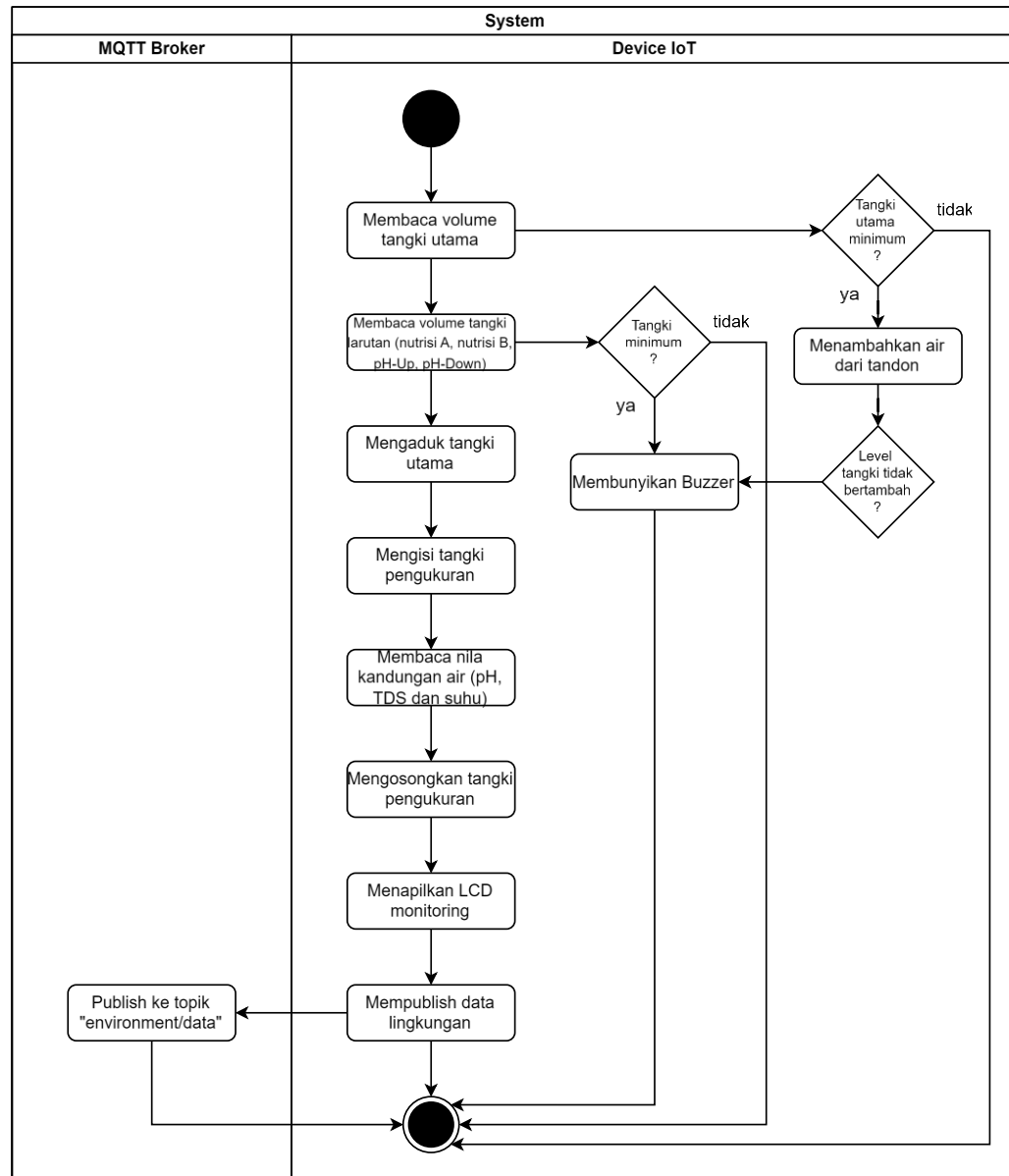
Pada Gambar 4.7, menyajikan *activity diagram* yang mengintegrasikan fungsi dari beberapa *use case* (UC08, UC03, UC10, dan UC07) terkait pemantauan dan notifikasi. Alur sistem dimulai saat *Device IoT* menginisiasi koneksi ke jaringan WiFi. Apabila koneksi gagal, perangkat akan terus mencoba menghubungkan ulang (*retry*). Setelah koneksi WiFi terbentuk, perangkat melanjutkan proses koneksi ke

MQTT Broker dengan mekanisme *retry* yang sama jika terjadi kegagalan. Setelah berhasil terhubung ke broker, perangkat segera memublikasikan statusnya ke topik “*device/status*”.

Sistem N8N memantau topik “*device/status*” untuk mendeteksi status perangkat. Jika perangkat terdeteksi *offline*, N8N akan mengirimkan *push notification* ke *Mobile App* disisi *User*. Sebaliknya, jika perangkat dalam kondisi online, N8N akan memicu pengambilan data secara berkala (setiap 30 menit) dengan memublikasikan pesan ke topik “*environment/refresh*”. Merespons perintah tersebut, *Device IoT* akan membaca data sensor lingkungan terkini dan memublikasikannya ke topik “*environment/data*”.

Selain pengiriman data rutin, *Device IoT* juga memvalidasi setiap tangki (utama, nutrisi A, nutrisi B, *pH-Up* dan *pH-Down*). Jika level tangki terdeteksi berada pada batas minimum, perangkat akan memublikasikan pesan peringatan ke topik “*tank/notification*”. Pesan ini kemudian diproses oleh N8N untuk diteruskan sebagai notifikasi ke *Mobile App*. Secara bersamaan, *Device IoT* juga akan mengaktifkan *buzzer* sebagai tanda peringatan lokal.

f. AC06 Membaca Data Lingkungan



Gambar 4.8. AC06 Membaca Data Lingkungan

Berdasarkan Gambar 4.8, merupakan *activity diagram* memvisualisasikan alur *use case* UC10 DAN UC09, mengambil data lingkungan. Alur sistem dimulai dengan prosedur pengecekan ketersediaan cairan pada setiap tangki yang berjalan beriringan dengan inisiasi pengukuran. Sistem memantau volume air di tangki utama; jika volume terdeteksi di bawah batas minimum, perangkat secara otomatis mengaktifkan mekanisme pengisian air dari tandon. Pada saat yang sama, terdapat validasi keamanan untuk proses ini: jika level air tidak bertambah pasca pengisian (mengindikasikan tandon kosong atau gangguan pompa), *buzzer* akan berbunyi

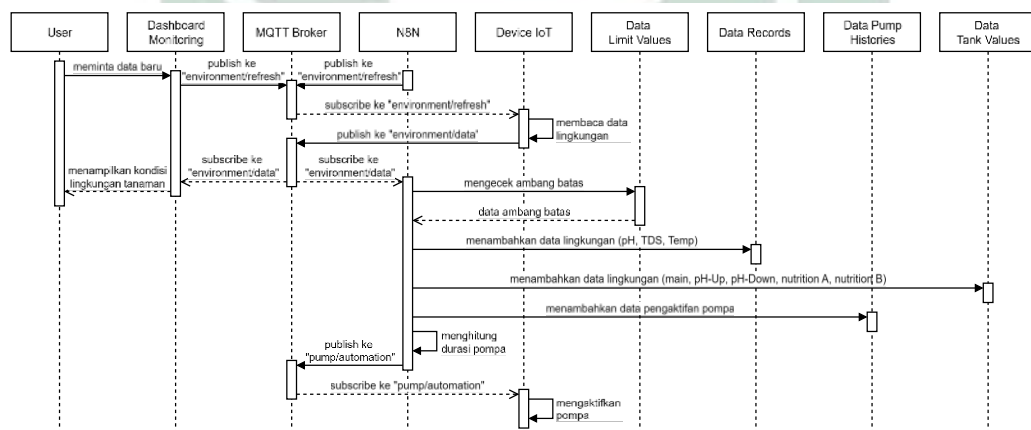
sebagai notifikasi peringatan tanpa mengganggu alur kerja sistem secara keseluruhan.

Secara paralel, sistem juga melakukan pemindaian terhadap volume tangki larutan (nutrisi A, nutrisi B, *pH-Up*, dan *pH-Down*). Apabila salah satu tangki larutan terdeteksi berada pada level minimum, sistem akan membunyikan *buzzer* untuk memberi tahu pengguna. Mekanisme pengisian otomatis maupun peringatan audio ini bersifat *non-blocking*, artinya proses tersebut terjadi di latar belakang sehingga proses utama pengambilan data lingkungan tetap akan dilanjutkan ke tahap berikutnya terlepas dari adanya notifikasi peringatan level cairan.

Sistem kemudian masuk ke tahap inti pengukuran dengan terlebih dahulu mengaduk tangki utama guna memastikan larutan tercampur rata. Setelah proses pengadukan selesai, pompa akan mengisi tangki pengukuran dengan air sampel. Sensor kemudian membaca parameter kualitas air yang meliputi nilai *pH*, TDS, dan suhu. Setelah data berhasil didapatkan, tangki pengukuran dikosongkan kembali. Data tersebut kemudian ditampilkan secara lokal pada *LCD monitoring* dan, sebagai langkah terakhir, dipublikasikan ke topik “*environment/data*” melalui MQTT Broker untuk didistribusikan ke sistem lain, menandai selesainya satu siklus pembacaan data.

4.1.1.2.3. Sequence Diagram

a. SQ01 Kontrol Otomatis

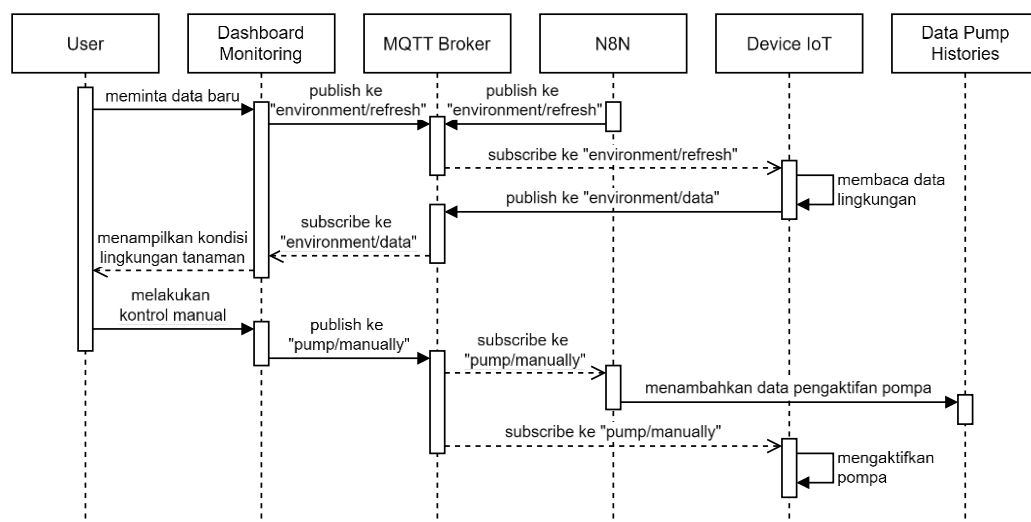


Gambar 4.9. SQ01 Kontrol Otomatis

Berdasarkan Gambar 4.9, *sequence diagram* ini memvisualisasikan interaksi antar objek dalam sistem untuk menangani alur pemantauan dan kontrol otomatis. Proses mulai oleh *User* yang meminta pembaruan data melalui *Dashboard Monitoring* atau N8N yang melakukan *trigger* dengan memublikasikan pesan ke topik “*environment/refresh*” melalui *MQTT Broker*. Pesan ini direspons oleh *Device IoT* untuk melakukan pembacaan data lingkungan dan mengirimkan hasilnya kembali ke topik “*environment/data*”. Data tersebut kemudian diteruskan ke dua arah ke *Dashboard Monitoring* untuk ditampilkan kepada pengguna, dan ke N8N untuk pemrosesan logika.

Di sisi *backend*, setelah N8N menerima data lingkungan, sistem melakukan pengecekan terhadap konfigurasi ambang batas pada *Data Limit Values* dan menyimpan data lingkungan (*pH*, *TDS* dan suhu) ke *Data Records* dan data lingkungan (tangki utama, nutrisi A, nutrisi B, *pH-Up* dan *pH-Down*) ke *Data Tank Levels*. Jika kondisi lingkungan memerlukan tindakan koreksi (berdasarkan ambang batas), N8N melakukan kalkulasi durasi pompa secara internal. Selanjutnya, sistem menyimpan riwayat tindakan tersebut ke dalam *Data Pump Histories* dan memublikasikan perintah eksekusi ke topik “*pump/automation*”. Alur ini diakhiri dengan *Device IoT* yang menerima perintah tersebut dan mengaktifkan pompa sesuai durasi yang telah ditentukan.

b. SQ02 Kontrol Manual via Aplikasi

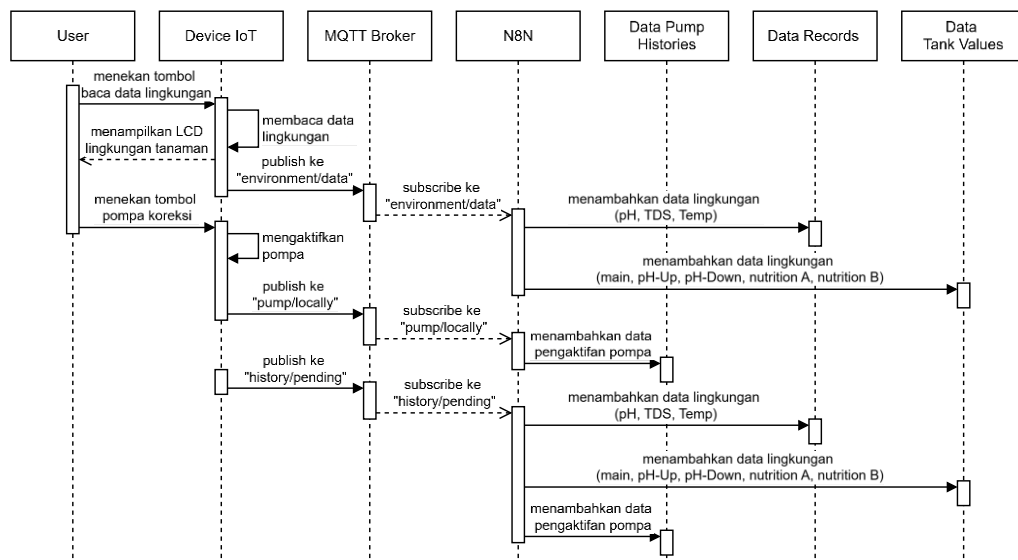


Gambar 4.10. SQ02 Kontrol Manual via Aplikasi

Berdasarkan Gambar 4.10, *sequence diagram* ini merincikan interaksi antar objek dalam sistem saat pengguna melakukan pemantauan dan pengendalian pompa secara manual melalui aplikasi. Alur dimulai dengan fase pemantauan, di mana *User* meminta pembaruan data melalui *Dashboard Monitoring* atau N8N yang melakukan *trigger* dengan memublikasikan pesan ke topik “*environment/refresh*” melalui *MQTT Broker*. Merespons perintah tersebut, *Device IoT* melakukan pembacaan data lingkungan dan mengirimkan hasilnya ke topik “*environment/data*”. Data tersebut kemudian diterima oleh *Dashboard Monitoring* untuk ditampilkan sebagai kondisi terkini tanaman kepada pengguna.

Fase kedua adalah eksekusi kontrol manual. Ketika *User* memberikan instruksi kontrol untuk menambahkan nutrisi atau *pH-Up/Down* melalui antarmuka aplikasi, *Dashboard Monitoring* memublikasikan perintah ke topik “*pump/manually*”. *MQTT Broker* mendistribusikan pesan tersebut secara paralel ke dua entitas tujuan: N8N dan *Device IoT*. N8N yang berlangganan ke topik tersebut bertugas mencatat aktivitas pengaktifan pompa ke dalam *Data Pump Histories* sebagai log sistem. Secara bersamaan, *Device IoT* menerima perintah yang sama dan langsung mengeksekusi pengaktifan pompa secara fisik sesuai instruksi.

c. SQ03 Kontrol Manual via Lokal



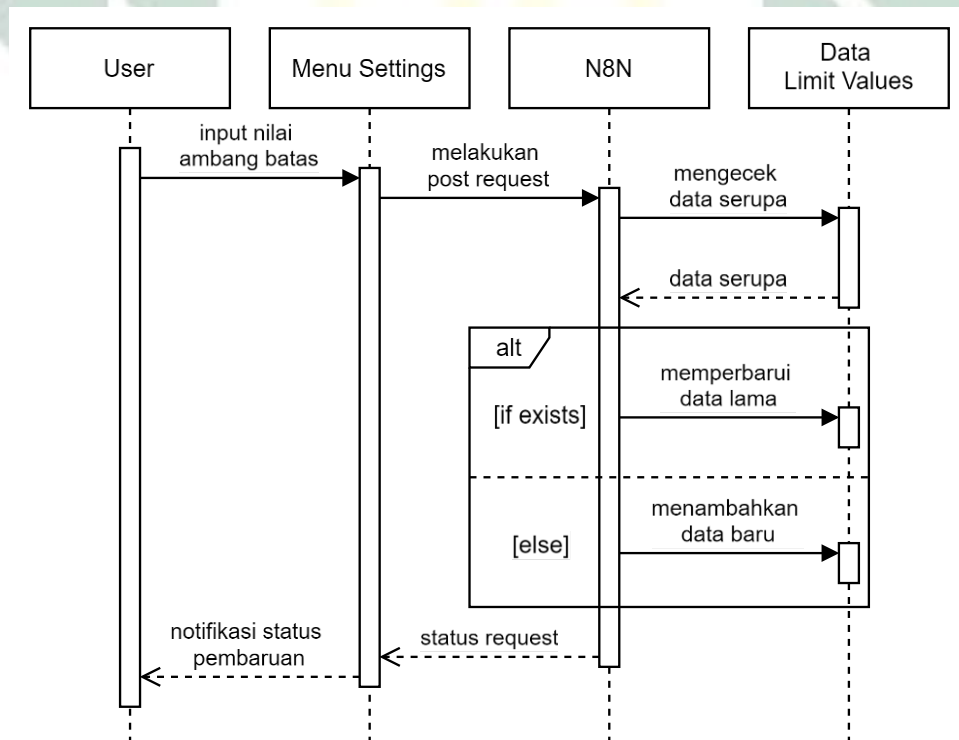
Gambar 4.11. SQ03 Kontrol Manual via Lokal

Berdasarkan Gambar 4.11, *sequence diagram* ini memvisualisasikan interaksi langsung antara pengguna dan perangkat keras tanpa melalui antarmuka aplikasi

seluler. Alur dimulai saat *User* menekan tombol baca data, yang direspons oleh *Device IoT* dengan membaca sensor internal dan menampilkan hasilnya ke LCD. Setelah data tampil, perangkat segera memublikasikan data tersebut ke topik “*environment/data*”. Pesan ini diproses oleh N8N untuk didistribusikan penyimpanannya ke dua entitas: parameter lingkungan (*pH*, TDS, Suhu) disimpan ke *Data Records*, sedangkan volume cairan disimpan ke *Data Tank Values*.

Pada tahap eksekusi kontrol, saat *User* menekan tombol pompa, *Device IoT* secara langsung mengaktifkan pompa fisik. Setelah eksekusi selesai, perangkat memublikasikan status aktivitas ke topik “*pump/locally*”, yang kemudian dicatat oleh N8N ke dalam *Data Pump Histories*. Diagram ini juga memperlihatkan mekanisme *fail-safe* melalui topik “*history/pending*”. Jika terjadi kegagalan pengiriman pada jalur utama, *Device IoT* akan mengirimkan paket data gabungan ke topik tersebut. N8N kemudian akan memecah dan menyimpan data tersebut secara simultan ke *Data Pump Histories*, *Data Records*, dan *Data Tank Values*, memastikan integritas data tetap terjaga meskipun terjadi kendala jaringan sesaat.

d. SQ04 Mengatur Ambang Batas

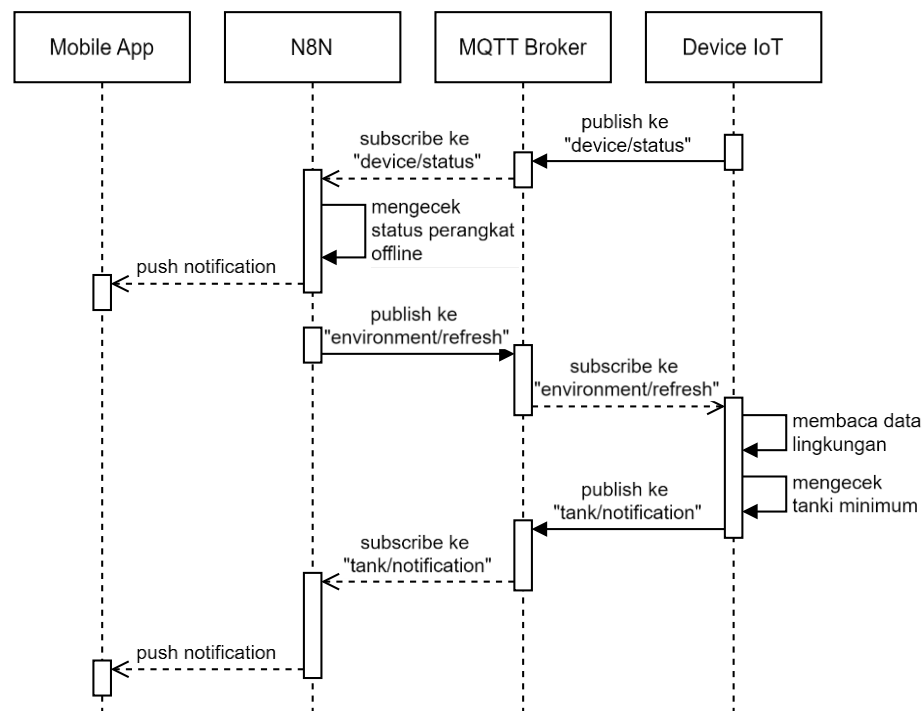


Gambar 4.12. SQ04 Mengatur Ambang Batas

Berdasarkan Gambar 4.12, *sequence diagram* ini merincikan alur pertukaran pesan antar objek saat pengguna mengonfigurasi parameter kontrol. Interaksi dimulai oleh *User* yang memasukkan nilai ambang batas (*pH* dan nutrisi) pada antarmuka *Menu Settings*. Aplikasi kemudian mengirimkan data tersebut ke server melalui *post request* yang ditujukan kepada N8N.

Setelah menerima permintaan, N8N melakukan validasi dengan memeriksa keberadaan data yang identik pada *Data Limit Values*. Proses ini melibatkan logika percabangan; jika data serupa ditemukan, sistem akan memperbarui data lama dengan mengubah statusnya menjadi aktif. Sebaliknya, jika data belum tersedia, sistem akan menyimpan masukan tersebut sebagai data baru. Setelah proses *database* selesai, N8N mengembalikan status permintaan ke *Menu Settings*, yang selanjutnya menampilkan notifikasi status pembaruan kepada *User* sebagai konfirmasi bahwa konfigurasi telah berhasil disimpan.

e. SQ05 Sistem Notifikasi



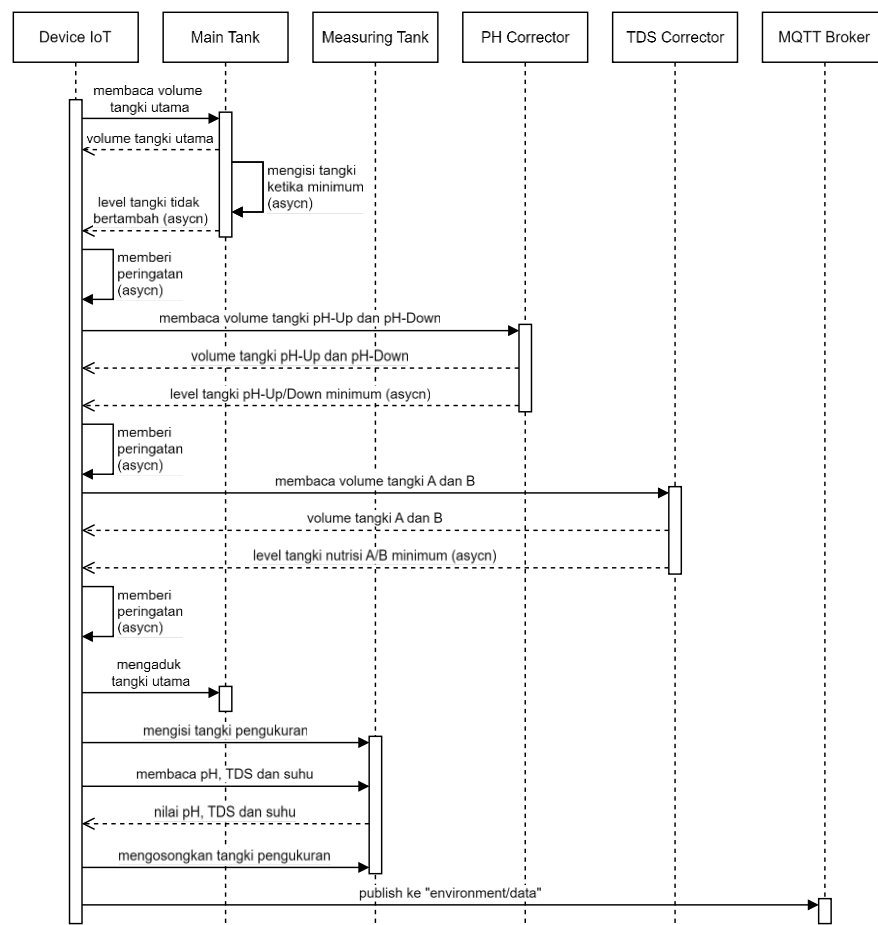
Gambar 4.13. SQ05 Sistem Notifikasi

Berdasarkan Gambar 4.13, *sequence diagram* ini memvisualisasikan interaksi antar objek dalam menangani peringatan dan notifikasi sistem. Alur dimulai dengan *Device IoT* yang melaporkan status konektivitasnya dengan memublikasikan pesan

ke topik “*device/status*” melalui *MQTT Broker*. N8N, yang berlangganan topik tersebut, menerima data status dan melakukan pengecekan. Apabila perangkat terdeteksi dalam kondisi *offline*, N8N akan segera mengirimkan *push notification* ke *Mobile App* untuk memberi tahu pengguna.

Di sisi lain, saat siklus pembaruan data berjalan, N8N memublikasikan perintah ke topik “*environment/refresh*”. *Device IoT* merespons perintah ini dengan melakukan dua aktivitas internal: membaca data lingkungan dan mengecek level air tangki. Jika hasil pengecekan menunjukkan volume air mencapai batas minimum, *Device IoT* memublikasikan peringatan ke topik “*tank/notification*”. Pesan ini kemudian diterima oleh N8N dan diteruskan kepada pengguna dalam bentuk *push notification* melalui *Mobile App*, memastikan pengguna segera mengetahui kondisi kritis pada sistem hidroponik.

f. SQ06 Membaca Data Lingkungan

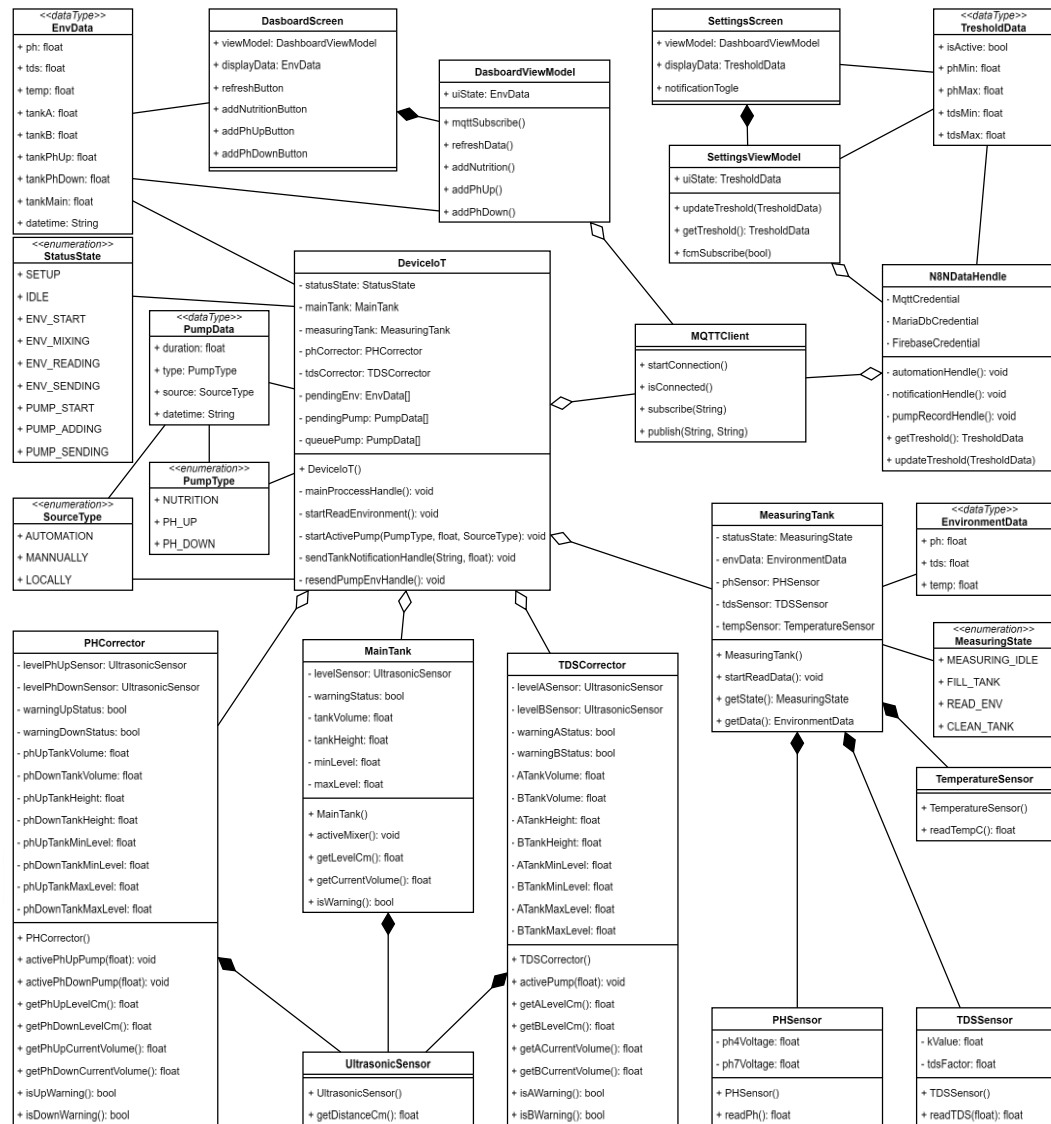


Gambar 4.14. SQ06 Membaca Data Lingkungan

Berdasarkan Gambar 4.14, *sequence diagram* ini merincikan interaksi antara *controller center (Device IoT)* dengan berbagai modul selama siklus pengambilan data. Alur diawali dengan serangkaian pengecekan volume cairan yang dilakukan secara paralel (*asynchronous*). *Device IoT* memanggil fungsi pada modul *Main Tank* untuk membaca volume tangki utama, serta modul korektor (PH Corrector dan TDS Corrector) untuk membaca masing-masing tangki koreksi. Jika terdeteksi kondisi kritis—seperti level air minimum pada tangki koreksi atau kegagalan pengisian air pada tangki utama—perangkat akan mengirimkan pesan peringatan (*warning*), namun proses ini didesain agar tidak memblokir alur utama sistem.

Setelah fase pengecekan selesai, sistem melanjutkan ke prosedur pengambilan sampel. *Device IoT* memanggil fungsi untuk melakukan pengadukan pada modul *Main Tank* agar larutan homogen. Selanjutnya, perangkat memanggil fungsi pengisian air ke dalam tangki pengukuran dengan modul *Measuring Tank*. Di dalam tangki pengukuran sensor-sensor melakukan pembacaan data lingkungan (*pH*, TDS, dan suhu). Setelah data lingkungan didapatkan, sistem mengosongkan kembali tangki pengukuran. Siklus ini ditutup dengan *Device IoT* memublikasikan hasil pembacaan data lingkungan tersebut ke topik “*environment/data*” melalui MQTT Broker untuk didistribusikan ke layanan lain.

4.1.1.2.4. Class Diagram



Gambar 4.15. Class Diagram

Class Diagram Gambar 4.15, menggambarkan arsitektur perangkat lunak untuk sistem *IoT* hidroponik, yang dirancang untuk mengintegrasikan aplikasi seluler dengan perangkat keras tertanam (*embedded system*). Secara garis besar, sistem ini bertujuan untuk memantau dan mengontrol kualitas air secara otomatis. Struktur diagram memisahkan tanggung jawab antara antarmuka pengguna, logika kontrol perangkat, manajemen sensor, dan lapisan komunikasi, memastikan setiap komponen memiliki peran yang terdefinisi dengan jelas dalam ekosistem otomatis.

Pada sisi aplikasi pengguna, sistem menerapkan pola desain *MVVM* (*Model-View-ViewModel*) untuk memisahkan logika tampilan dari pengolahan data. Hal ini

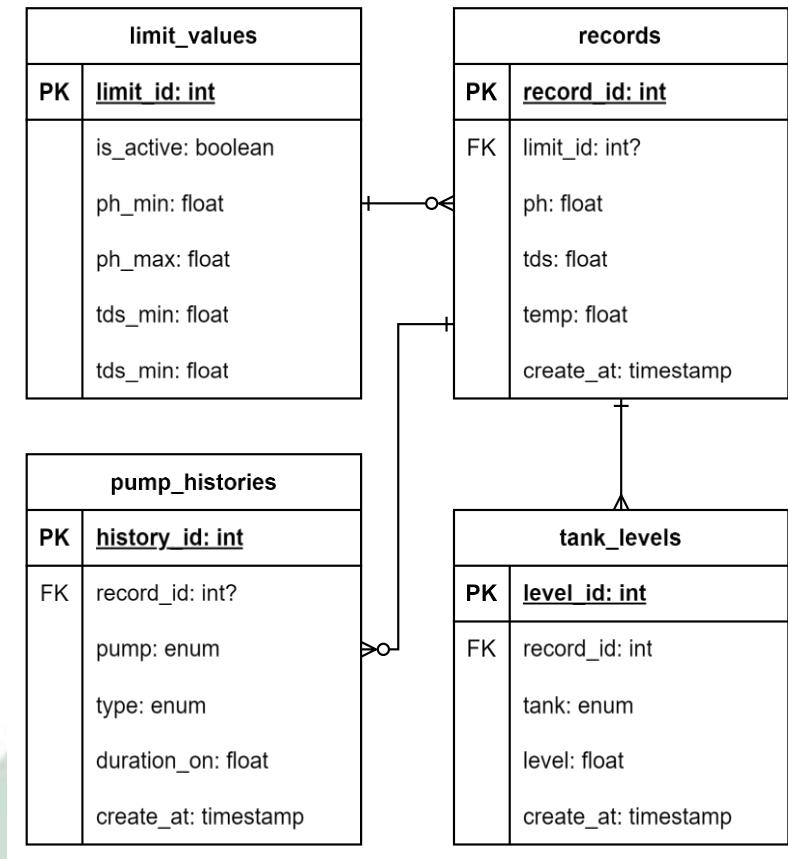
terlihat dari adanya kelas *DashboardScreen* dan *SettingsScreen* yang masing-masing didukung oleh *DashboardViewModel* dan *SettingsViewModel*. Melalui antarmuka ini, pengguna dapat melihat data lingkungan (seperti *pH*, TDS, suhu dan volume setiap tangki) yang tersimpan dalam objek *EnvData*, serta mengatur parameter otomatisasi melalui *ThresholdData*. Aplikasi ini juga memungkinkan pengguna untuk memberikan perintah manual, seperti menyalakan pompa nutrisi atau mengaktifkan pompa *pH*.

Jantung dari operasional perangkat keras dikendalikan oleh kelas pusat bernama *DeviceIoT*. Kelas ini bertindak sebagai kontrol utama yang mengatur status mesin (melalui *enum StatusState*) dan mengelola antrian tugas pompa. *DeviceIoT* memiliki hubungan komposisi dengan berbagai *subsistem* fisik, termasuk *MainTank* sebagai penampung utama dan *MeasuringTank* yang berfungsi khusus untuk melakukan pembacaan sensor.

Untuk pengendalian nutrisi dan *pH*, sistem menggunakan kelas *PHCorrector* dan *TDSCorrector*. Kedua kelas ini tidak hanya mengontrol pompa larutan koreksi, tetapi juga dilengkapi dengan *UltrasonicSensor* untuk memantau sisa volume cairan korektor di wadah persediaan. Sementara itu, proses pembacaan data air ditangani oleh kelas *MeasuringTank* yang mengagregasi data dari *PHSensor*, *TDSSensor*, dan *TemperatureSensor*, memastikan data yang dikirim ke pengguna adalah data yang akurat dan terkalibrasi.

Seluruh pertukaran data antara aplikasi dan perangkat keras dijembatani oleh protokol komunikasi yang dikelola oleh kelas *MQTTClient*. Kelas ini memungkinkan fitur *real-time* melalui mekanisme *publish* dan *subscribe*. Selain itu, terdapat kelas *N8NDataHandle* yang berfungsi sebagai penghubung ke layanan *backend* dan *database*. Kelas *N8NDataHandle* ini menangani kredensial untuk MariaDB dan Firebase, serta memfasilitasi logika otomatisasi tingkat lanjut yang mungkin diproses di server menggunakan *n8n*, seperti penyimpanan riwayat data pompa atau pemicu notifikasi darurat.

4.1.1.2.5. Entity-Relationship Diagram



Gambar 4.16. Entity-Relationship Diagram

Berdasarkan Gambar 4.16, sistem memiliki empat entitas utama: *limit_values*, *records*, *tank_levels*, dan *pump_histories*. Entitas *records* berfungsi sebagai pusat pencatatan data sensor *ph*, nutrisi (*tds*), *temperature* (*temp*). Setiap data yang tercatat dalam *records* terhubung ke entitas *limit_values* yang menyimpan ambang batas yang berlaku saat itu. Selain itu, setiap *record* juga terhubung ke *tank_levels* yang mencatat ketinggian air di dalam tangki dan *pump_histories* yang mencatat riwayat aktivitas pompa. Dengan demikian, arsitektur *database* ini memungkinkan pelacakan historis yang lengkap mengenai kondisi sistem, tindakan yang diambil, dan aturan yang diterapkan pada satu waktu tertentu.

4.1.1.3. Fase Implementasi

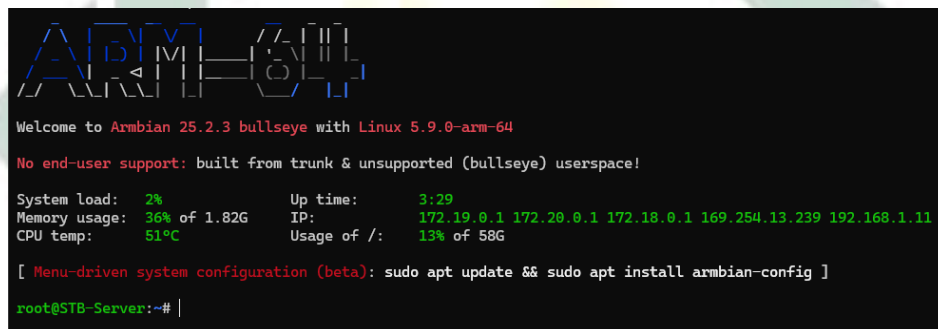
Sebagai bagian dari dokumentasi dan untuk mempermudah proses replikasi sistem, keseluruhan berkas konfigurasi yang diimplementasikan pada server, termasuk skenario *workflow* dan berkas Docker Compose, telah disimpan pada

repositories daring. Seluruh berkas pendukung tersebut dapat diakses secara publik melalui platform GitHub pada tautan berikut: <https://github.com/Anomali99/MyHydroponic-Workflow>.

4.1.1.3.1. Menyiapkan Server

a. Instalasi Armbian

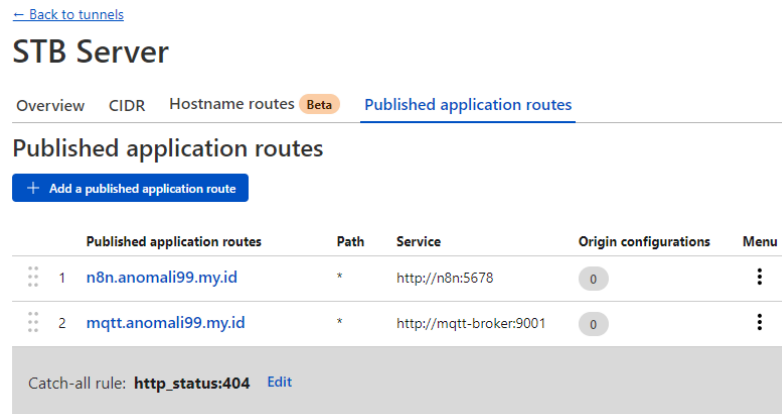
Tahap awal implementasi dimulai dengan persiapan sisi server, yaitu melakukan instalasi sistem operasi pada perangkat STB. Sistem operasi yang digunakan dalam penelitian ini adalah Armbian Server 25.2.3 (Bullseye) dengan kernel Linux 5.9.0-arm-64, yang diperoleh dari repositori komunitas pengembang. Proses instalasi dilakukan dengan cara menyalin (*burning*) *image* sistem operasi ke dalam *SD Card* menggunakan perangkat lunak Balena Etcher. Setelah proses *booting* berhasil dan konfigurasi awal selesai dilakukan, sistem akan menampilkan *CLI (Command Line Interface)* yang menandakan server siap digunakan, sebagaimana terlihat pada Gambar 4.17.



Gambar 4.17. Tampilan Awal Armbian

b. Menyiapkan *Cloudflare Tunnel*

Setelah konfigurasi dasar server selesai, tahap selanjutnya adalah mengimplementasikan *Cloudflare Tunnel* untuk mengekspos layanan lokal ke jaringan internet secara aman tanpa perlu membuka *port* pada *router* publik. Proses ini melibatkan pembuatan *tunnel* dan autentikasi server menggunakan token yang diberikan oleh penyedia layanan. Dalam penelitian ini, dikonfigurasi dua rute aplikasi utama, yaitu `n8n.anomali99.my.id` yang diarahkan ke layanan `n8n` pada *port* 5678, serta `mqtt.anomali99.my.id` yang ditujukan untuk komunikasi ke *MQTT Broker* melalui protokol *WebSocket* pada *port* 9001. Konfigurasi rute aplikasi yang telah diimplementasikan dapat dilihat pada Gambar 4.18.



Gambar 4.18. Cloudflare Tunnel Routes

c. Menyiapkan Layanan Docker

Tahap selanjutnya dalam pengembangan sistem *backend* adalah melakukan instalasi Docker Engine pada perangkat STB server. Platform ini digunakan untuk menjalankan aplikasi dalam lingkungan yang terisolasi. Beberapa kontainer utama yang disiapkan meliputi Cloudflared, n8n, MariaDB, dan MQTT Broker. Kontainer Cloudflared bertugas membangun koneksi *tunnel* menggunakan token yang diperoleh sebelumnya, sementara n8n berfungsi sebagai *beckend* dan kontrol utama. Selain itu, disiapkan pula MariaDB sebagai sistem manajemen basis data untuk penyimpanan log data lingkungan maupun aktivitas pompa, serta MQTT Broker yang berperan sebagai jembatan komunikasi *real-time* antar-perangkat.

Seluruh layanan tersebut dikonfigurasi menggunakan Docker Compose untuk mempermudah dalam pemeliharaan sistem di masa mendatang. Dalam konfigurasi ini, diterapkan mekanisme *volume mapping* guna menjamin persistensi data, sehingga konfigurasi n8n, log MQTT, maupun data pada MariaDB tetap tersimpan aman meskipun kontainer mengalami restart. Lebih lanjut, didefinisikan pula sebuah jaringan internal (*private network*) yang menghubungkan keempat kontainer tersebut. Hal ini memastikan n8n dapat mengakses MariaDB dan MQTT Broker secara langsung melalui jaringan lokal kontainer yang aman dan terisolasi.

Sebagai langkah pengamanan tambahan pada layanan MQTT Broker, diterapkan mekanisme autentikasi untuk mencegah akses tanpa izin. Konfigurasi ini melibatkan pembuatan tiga kredensial pengguna yang terpisah, masing-masing dialokasikan secara spesifik untuk layanan n8n, aplikasi seluler, dan perangkat

microcontroller ESP32. Penerapan manajemen akses pengguna ini bertujuan untuk memastikan bahwa hanya entitas yang terverifikasi yang dapat terhubung ke broker, sehingga mengurangi risiko keamanan dari akses eksternal yang tidak diinginkan.

Berikut kode docker-compose.yml yang digunakan:

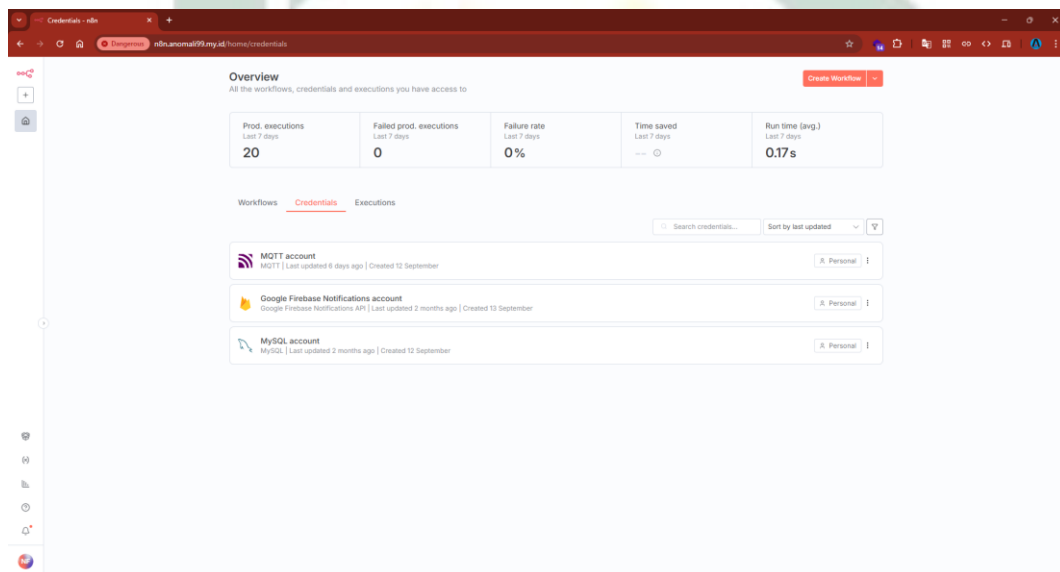
1	services:
2	cloudflared:
3	image: cloudflare/cloudflared:latest
4	container_name: cloudflared-tunnel
5	restart: unless-stopped
6	command: tunnel --no-autoupdate run --token TOKEN
7	networks:
8	- private-net
9	
10	n8n:
11	image: docker.n8n.io/n8nio/n8n:1.110.1
12	container_name: n8n
13	restart: unless-stopped
14	ports:
15	- "5678:5678"
16	environment:
17	- GENERIC_TIMEZONE=Asia/Jakarta
18	- TZ=Asia/Jakarta
19	- N8N_ENFORCE_SETTINGS_FILE_PERMISSIONS=true
20	- N8N_RUNNERS_ENABLED=true
21	- WEBHOOK_URL=https://n8n.anomali99.my.id/
22	networks:
23	- private-net
24	volumes:
25	- ./n8n/data:/home/node/.n8n
26	
27	mariadb:
28	image: mariadb:10.5

29	container_name: mariadb
30	restart: unless-stopped
31	environment:
32	- MARIADB_ROOT_PASSWORD=
33	- MARIADB_USER=
34	- MARIADB_PASSWORD=
35	volumes:
36	- mariadb:/var/lib/mysql
37	- /etc/localtime:/etc/localtime:ro
38	networks:
39	- private-net
40	ports:
41	- "3306:3306"
42	
43	mqtt-broker:
44	image: eclipse-mosquitto:latest
45	container_name: mqtt-broker
46	restart: unless-stopped
47	ports:
48	- "1883:1883"
49	- "9001:9001"
50	networks:
51	- private-net
52	volumes:
53	- ./mosquitto/config:/mosquitto/config
54	- ./mosquitto/data:/mosquitto/data
55	- ./mosquitto/log:/mosquitto/log
56	
57	volumes:
58	mariadb:
59	external: true
60	
61	networks:

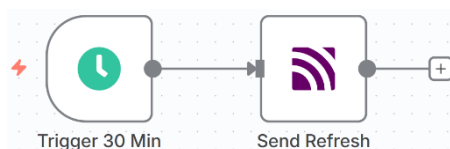
62	private-net:
63	external: true

d. Membuat *Workflow* N8N

Tahap selanjutnya adalah perancangan alur kerja (*workflow*) pada platform n8n. Dalam penelitian ini, logika sistem dibangun menggunakan kombinasi antara *node* bawaan dan *node* komunitas. Khusus untuk layanan notifikasi *push* ke aplikasi seluler, ditambahkan modul komunitas dengan nama paket `@digital-boss/n8n-nodes-google-firebase-notifications` yang terintegrasi dengan *Firebase Cloud Messaging (FCM)*. Agar interaksi antar-layanan dapat berjalan, diperlukan konfigurasi autentikasi sistem. Sebagaimana ditampilkan pada Gambar 4.19, telah disiapkan tiga kredensial utama, yaitu kredensial untuk koneksi ke MQTT Broker, *Google Firebase Notifications*, dan *database* MariaDB (menggunakan tipe akun MySQL).



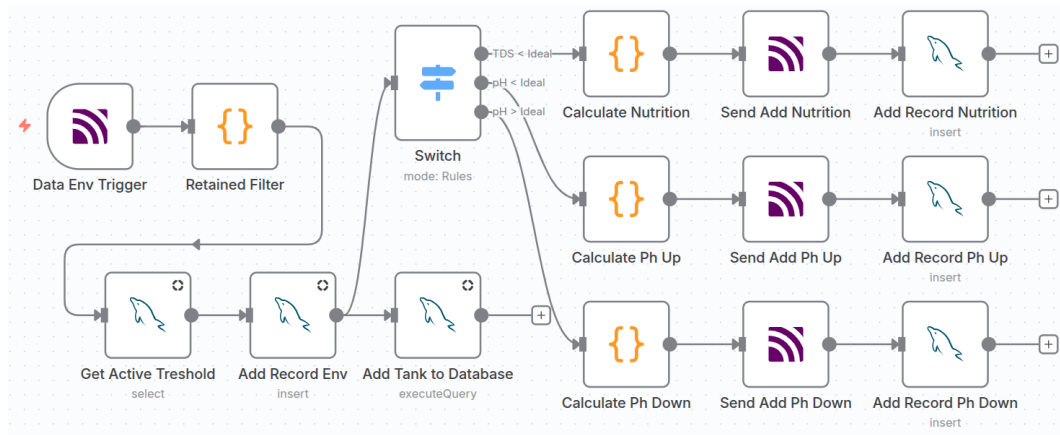
Gambar 4.19. Credentials N8N



Gambar 4.20. Workflow Trigger 30 Menit

Gambar 4.20 menunjukkan implementasi *workflow* yang dirancang untuk berjalan secara otomatis dengan interval waktu 30 menit. Alur kerja ini berfungsi

memicu sistem untuk menerbitkan pesan ke topik MQTT “*environment/refresh*”, yang bertujuan meminta pembaruan data terkini dari perangkat.



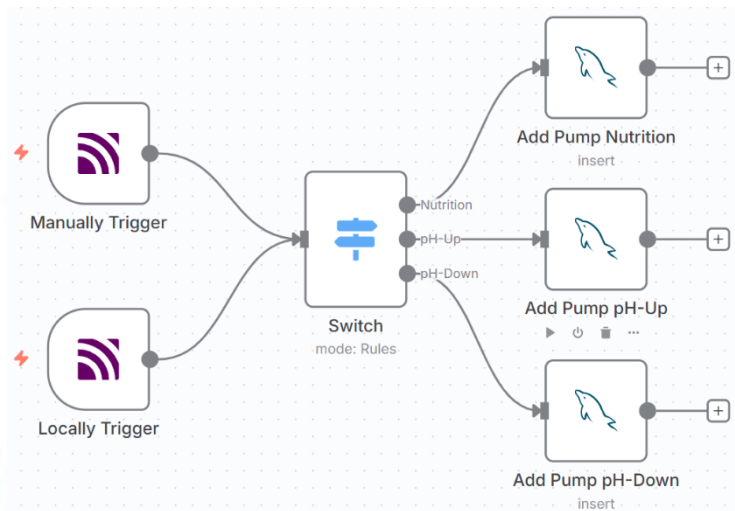
Gambar 4.21. Workflow Kontrol Otomatis

Implementasi mekanisme kontrol otomatis diperlihatkan pada Gambar 4.21. Saat data masuk ke sistem melalui topik “*environment/data*”, aliran proses diarahkan melewati *node Retained Filter* sebagai mekanisme validasi awal. *Node* ini bertugas menyaring pesan masuk untuk memastikan bahwa data yang diproses adalah data baru, bukan merupakan pesan *retained* (data lama yang tertahan di broker). Hal ini penting untuk mencegah sistem melakukan eksekusi aksi berdasarkan data usang saat terjadi koneksi ulang.

Setelah data sensor diterima, *workflow* mengambil nilai ambang batas aktif dari basis data menggunakan *node Get Active Threshold*. Selanjutnya, data parameter lingkungan dan status volume tangki saat ini disimpan ke dalam *database* melalui *node Add Record Env* dan *Add Tank to Database*. Data tersebut kemudian dievaluasi oleh *node Switch* untuk menentukan kondisi air berdasarkan aturan berikut: (1) TDS kurang dari ideal, (2) *pH* kurang dari ideal, atau (3) *pH* lebih dari ideal.

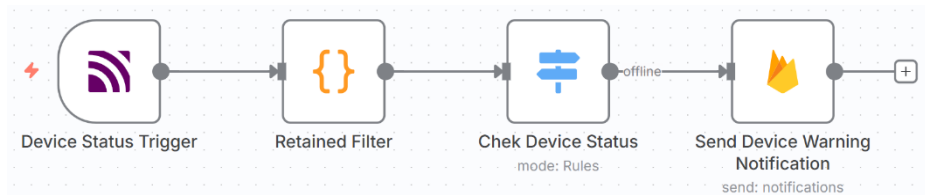
Berdasarkan hasil evaluasi kondisi air, sistem melakukan kalkulasi durasi pompa pada *node Calculate* yang relevan. Perhitungan pada *node Calculate Nutrition* menerapkan Rumus 3.2 dan Rumus 3.3, dengan parameter: *TT* sebagai rata-rata TDS ideal, *CT* sebagai nilai bacaan TDS sensor, *NT* bernilai 50.000 ppm, serta *FL_N* sebesar 40 ml/s. Sementara itu, *node Calculate pH Up* dan *pH Down* menggunakan Rumus 3.4 dan Rumus 3.5, dengan parameter dosis penyesuaian *R_{rec}*

sebesar 0,1 ml/L dan FL_{pH} sebesar 40 ml/s. Perlu dicatat bahwa perhitungan volume air ($V_{current}$) sesuai Rumus 3.1 telah diimplementasikan langsung pada perangkat *IoT*, sehingga data yang diterima oleh server sudah dalam besaran volume. Setelah durasi dihitung, sistem mengirimkan perintah aksi ke topik “*pump/automation*” melalui MQTT dan menyimpan riwayat tindakan penyesuaian tersebut ke dalam *database* menggunakan *node Add Record Action*.



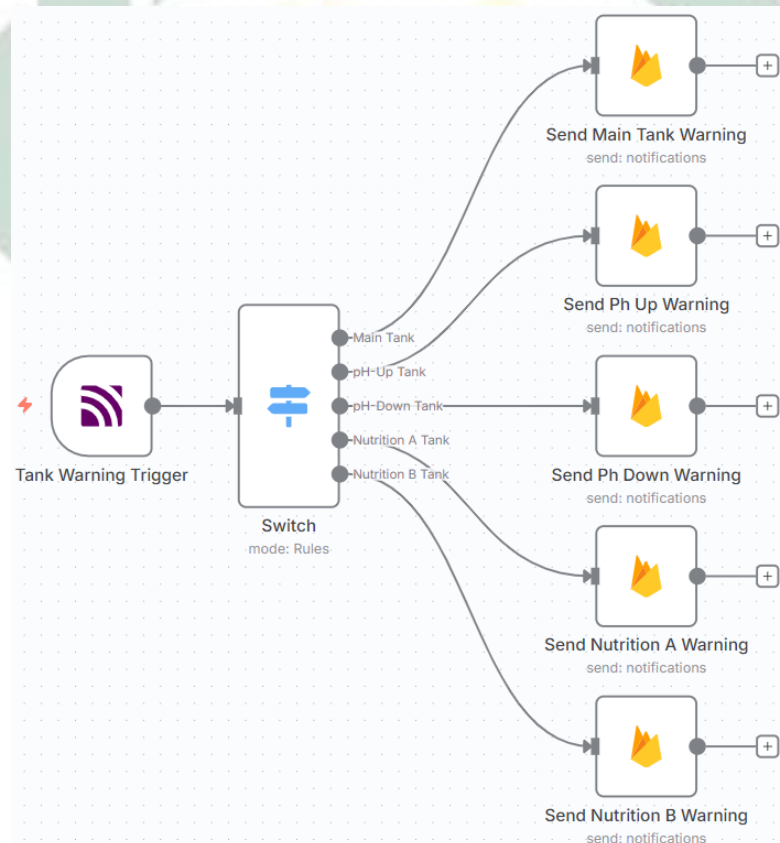
Gambar 4.22. Workflow untuk Aktivasi Pompa Manual via Aplikasi atau Lokal

Selanjutnya implementasi *workflow* untuk aktivasi pompa manual baik melalui aplikasi maupun secara lokal, Gambar 4.22. *Workflow* ini dimulai dengan mendeteksi input dari dua sumber MQTT, yaitu *node* “*Manually Trigger*” untuk perintah yang dikirim dari antarmuka aplikasi dan *node* “*Locally Trigger*” untuk interaksi fisik di perangkat lokal. Sinyal perintah yang diterima kemudian diproses oleh *node* “*Switch*”, yang bertugas menyeleksi dan mengarahkan aliran data berdasarkan parameter tipe pompa yang dituju, yaitu jalur untuk *Nutrition*, *pH-Up*, atau *pH-Down*. Setelah jalur ditentukan, alur kerja akan mengeksekusi *node database (insert)* yang sesuai—“*Add Pump Nutrition*”, “*Add Pump pH-Up*”, atau “*Add Pump pH-Down*”—untuk mencatat riwayat aktivasi manual tersebut ke dalam sistem *database* sebagai log aktivitas.



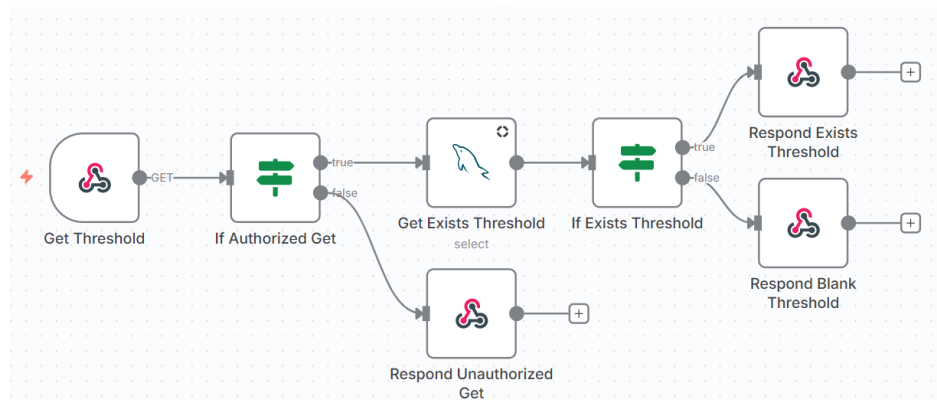
Gambar 4.23. Workflow Sistem Notifikasi Status Perangkat

Selanjutnya implementasi *workflow* sistem notifikasi status perangkat, Gambar 4.23. *Workflow* dimulai dari “*Device Status Trigger*” yang memantau konektivitas perangkat keras. Data yang masuk dari pemicu ini terlebih dahulu disaring oleh *node Retained Filter*. Mekanisme ini diterapkan untuk memastikan bahwa notifikasi status perangkat hanya dipicu oleh perubahan kondisi aktual, serta mencegah peringatan palsu akibat pembacaan pesan tertahan (*retained message*) saat memulai sistem. Setelah lolos validasi, status diperiksa oleh *node “Check Device Status”* sebelum dikirimkan peringatan melalui *node “Send Device Warning Notification”*.



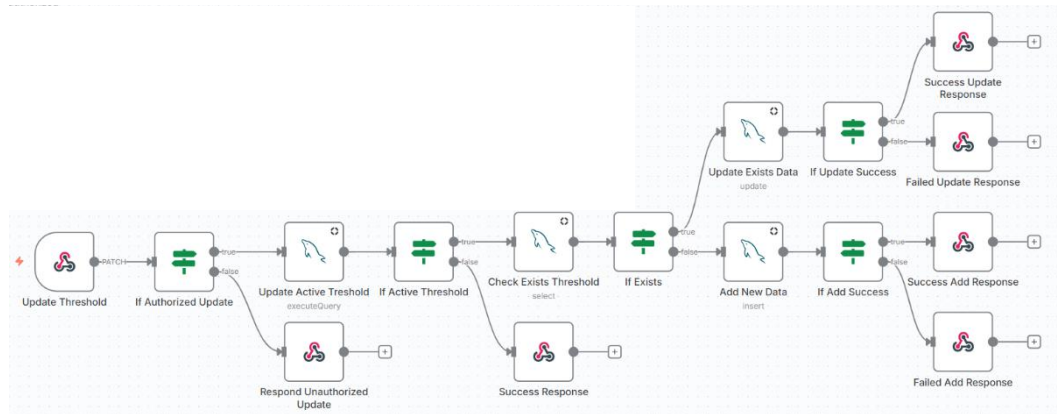
Gambar 4.24. Workflow Sistem Notifikasi Level Cairan

Selain itu ada *workflow* untuk sistem notifikasi level cairan, Gambar 4.24. Alur dimulai dari “*Tank Warning Trigger*”, yang mendeteksi level kritis pada tangki. Data dari pemicu ini diproses oleh *node* “*Switch*” yang memilah peringatan berdasarkan sumber tangki yang kritis, yaitu: *Main Tank*, *pH-Up Tank*, *pH-Down Tank*, *Nutrition A Tank*, serta *Nutrition B Tank*. Setiap kondisi tersebut akan mengaktifkan *node* pengiriman notifikasi spesifik (seperti “*Send Main Tank Warning*”, dst.) agar pengguna segera mengetahui tangki yang perlu diisi ulang.



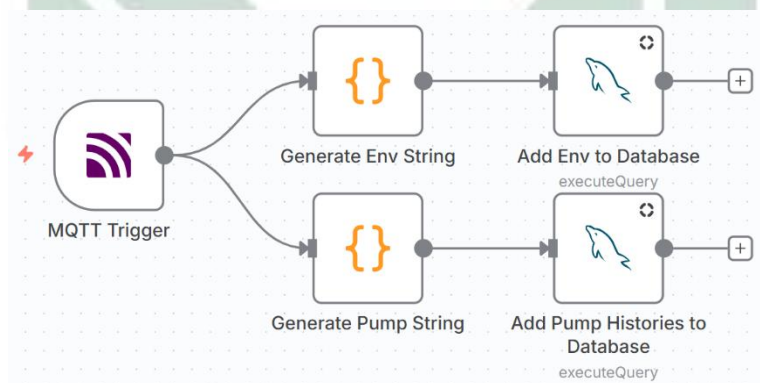
Gambar 4.25. Workflow API untuk Mengambil Ambang Batas

Gambar 4.25 memperlihatkan implementasi *workflow* API yang dikhususkan untuk proses pengambilan data konfigurasi. Alur kerja dipicu oleh permintaan HTTP dengan metode GET pada *endpoint webhook* yang telah ditentukan. Tahap pertama adalah validasi keamanan melalui *node If Authorized Get*, yang bertugas memverifikasi hak akses pemanggil. Jika otorisasi dinyatakan valid, sistem melanjutkan proses dengan mengeksekusi *node Get Exists Threshold* untuk mengambil data konfigurasi aktif dari *database*. Selanjutnya, aliran data melewati logika pengecekan kondisi pada *node If Exists Threshold*. Apabila data ditemukan, sistem akan mengirimkan respons JSON berisi nilai ambang batas melalui *node Respond Exists Threshold*; sebaliknya, jika data tidak tersedia, sistem memberikan balasan *default* melalui *node Respond Blank Threshold*.



Gambar 4.26. Workflow API untuk Mengatur Ambang Batas

Sementara itu, Gambar 4.26 mendetailkan mekanisme *workflow* API untuk memperbarui atau mengatur ulang nilai ambang batas menggunakan metode PATCH. Setelah permintaan diterima dan lolos validasi pada node *If Authorized Update*, sistem melakukan langkah awal dengan memperbarui status konfigurasi lama menjadi tidak aktif menggunakan node *Update Active Threshold*. Proses kemudian berlanjut ke node *Check Exists Threshold* untuk memeriksa ketersediaan data ambang batas dalam *database*. Berdasarkan hasil pemeriksaan ini, alur kerja terbagi menjadi dua cabang logika: jika data sudah ada, sistem melakukan pembaruan nilai melalui node *Update Exists Data*; namun jika belum tersedia, sistem akan menyisipkan data baru menggunakan node *Add New Data*. Seluruh rangkaian proses ini ditutup dengan pengiriman status akhir operasi—baik sukses maupun gagal—kembali ke pemanggil API sebagai konfirmasi.



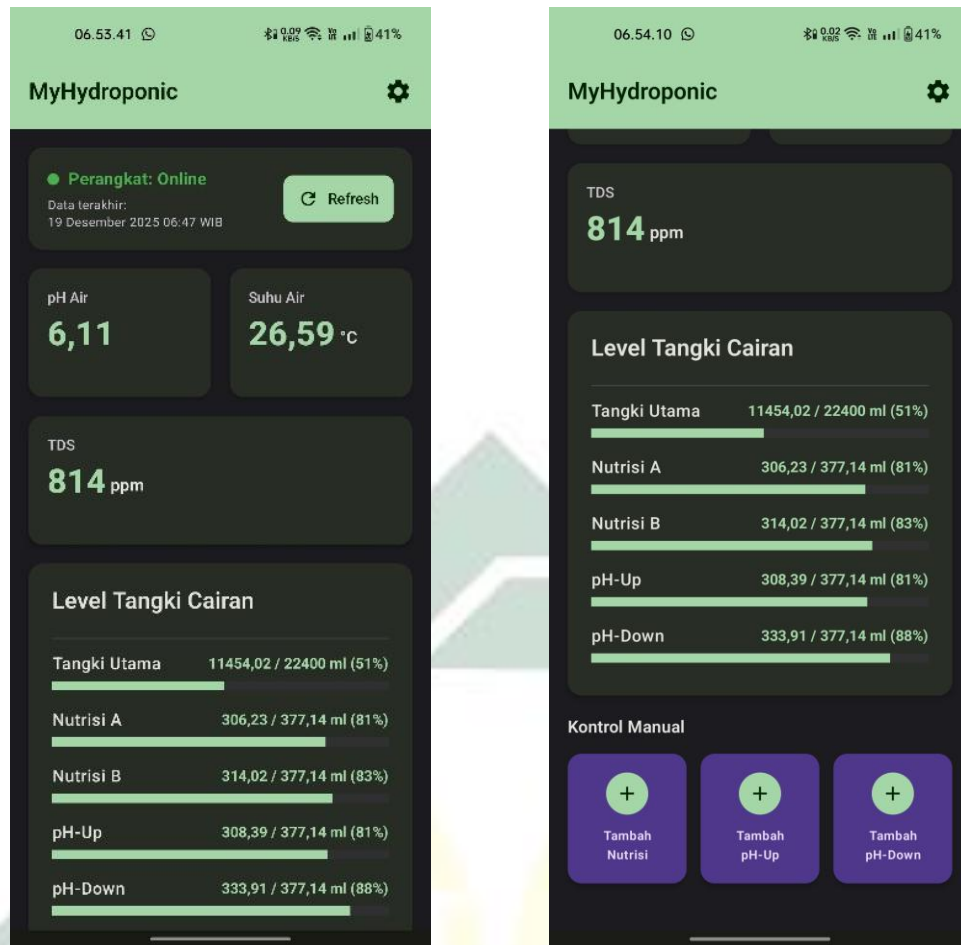
Gambar 4.27. Workflow untuk Pengiriman Data Lingkungan atau Pompa yang Tertunda

Dan yang terakhir *workflow* untuk pengiriman data lingkungan atau pompa yang tertunda, Gambar 4.27. *Workflow* ini dirancang untuk menangani data yang sebelumnya tertunda di penyimpanan lokal perangkat, biasanya akibat gangguan

koneksi internet. Proses dimulai ketika *node “MQTT Trigger”* menerima *payload* data yang dikirimkan secara *batch* dari perangkat keras saat koneksi kembali stabil. Alur kerja kemudian memproses data tersebut dalam dua jalur eksekusi paralel. Jalur atas bertugas menangani data sensor lingkungan, di mana *node “Generate Env String”* memformat string data masuk agar sesuai dengan skema *database*, lalu *node “Add Env to Database”* menyimpannya. Secara bersamaan, jalur bawah menangani log aktivitas pompa melalui *node “Generate Pump String”* untuk pemrosesan data dan *node “Add Pump Histories to Database”* untuk penyimpanan, sehingga menjamin integritas dan kelengkapan data historis sistem.

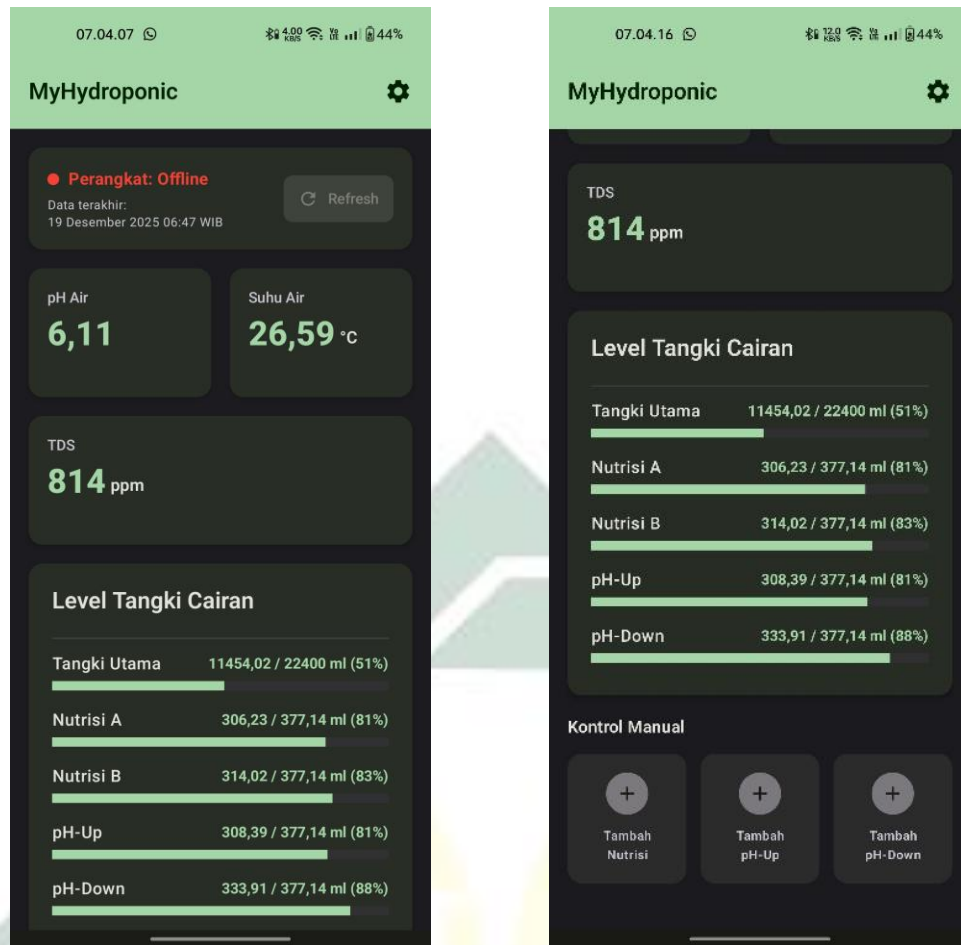
4.1.1.3.2. Membangun *Mobile App*

Mobile App MyHydroponic dibangun menggunakan Android Studio. Bahasa pemrograman yang dipilih adalah Kotlin yang dipadukan dengan *toolkit UI* modern Jetpack Compose. Dalam pengembangannya, aplikasi ini dikonfigurasi menggunakan rentang API Level SDK mulai dari versi 26 (Android 8.0 Oreo) hingga versi 36 (Android 16 Baklava). Struktur proyek mengadopsi arsitektur *MVVM (Model-View-ViewModel)* untuk memisahkan logika bisnis dari lapisan tampilan, serta dirancang untuk mendukung komunikasi data secara *real-time* dengan perangkat keras melalui protokol jaringan yang telah ditentukan. Sebagai bentuk transparansi dan dokumentasi pengembangan, keseluruhan kode sumber (*source code*) dari aplikasi ini dapat diakses secara publik melalui repositori GitHub pada tautan berikut: <https://github.com/Anomali99/MyHydroponic-App>.



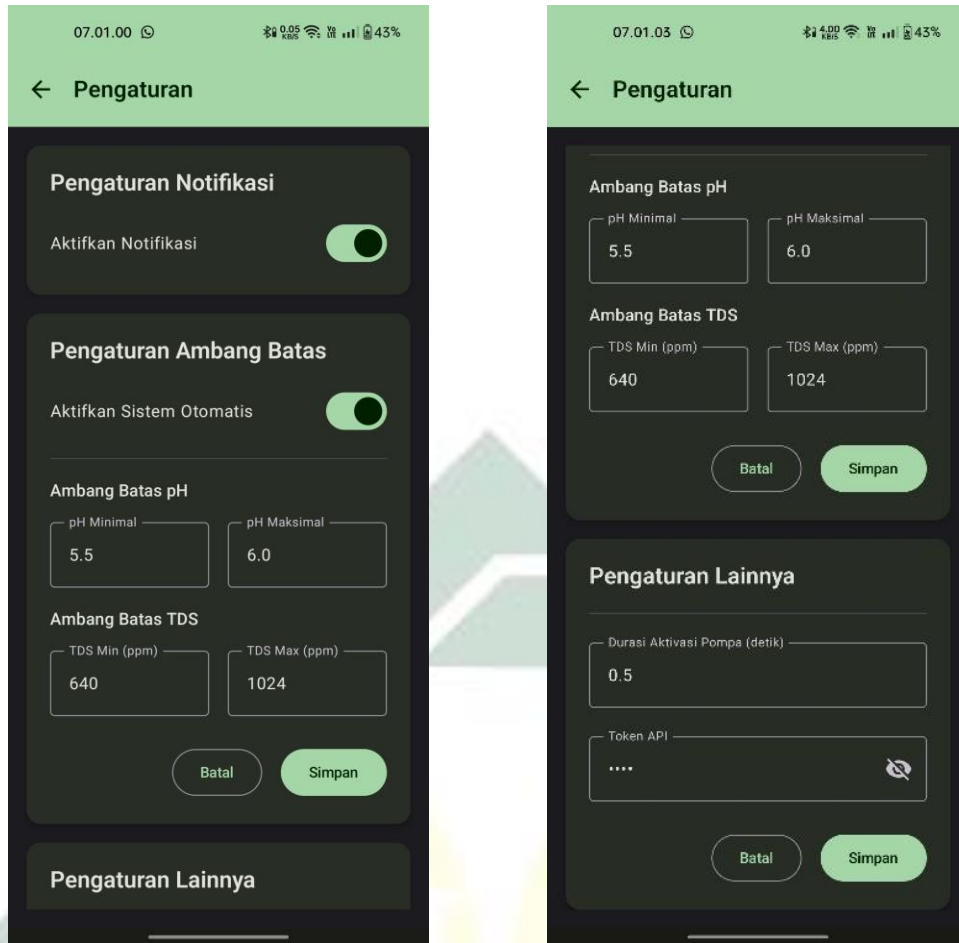
Gambar 4.28. Tampilan Dashboard Monitoring ketika Perangkat Online

Gambar 4.28 merupakan tampilan *Dashboard* aplikasi ketika sistem berhasil menjalin koneksi dengan perangkat *IoT*. Pada kondisi ini, indikator status di bagian atas layar secara jelas menampilkan label “Perangkat: *Online*” dengan ikon berwarna hijau, yang mengonfirmasi bahwa perangkat *IoT* terhubung broker. Halaman ini menyajikan informasi mengenai parameter lingkungan hidroponik, mulai dari pembacaan nilai sensor *pH* air, TDS, dan suhu air, hingga visualisasi volume cairan dalam berbagai tangki melalui indikator *progress bar*. Selain fungsi pemantauan, halaman ini juga memfasilitasi interaksi pengguna melalui tombol kontrol manual yang terletak di bagian bawah, memungkinkan aktivasi pompa nutrisi atau penyeimbang *pH* secara langsung apabila diperlukan intervensi di luar sistem otomatis.



Gambar 4.29. Tampilan Dashboard Monitoring ketika Perangkat Offline

Sebaliknya, Gambar 4.29 menampilkan antarmuka aplikasi ketika perangkat *IoT* terputus dari broker. Perubahan visual yang paling menonjol terlihat pada panel status yang berubah menjadi warna merah dengan keterangan “Perangkat: *Offline*”, memberikan peringatan visual instan kepada pengguna bahwa data yang ditampilkan adalah data terakhir yang diterima broker. Sebagai langkah preventif untuk menjaga validitas instruksi, sistem secara otomatis menonaktifkan (*disable*) seluruh elemen interaktif pada halaman ini. Hal ini mencakup tombol “*Refresh*” yang normalnya berfungsi untuk meminta pembaruan data, serta tombol-tombol kontrol manual di bagian bawah, sehingga pengguna tidak dapat mengirimkan perintah apa pun sampai perangkat *IoT* terhubung dengan broker.



Gambar 4.30. Tampilan Menu Settings

Gambar 4.30 menampilkan halaman pengaturan yang dirancang untuk memberikan fleksibilitas konfigurasi sistem kepada pengguna. Melalui antarmuka ini, pengguna dapat mengelola berbagai preferensi operasional, dimulai dari manajemen notifikasi untuk peringatan kondisi kritis hingga pengaturan ambang batas (*threshold*) kontrol otomatis. Pada bagian ambang batas, pengguna dapat menentukan nilai minimum dan maksimum untuk parameter *pH* dan TDS, yang mana nilai-nilai ini akan menjadi acuan bagi logika sistem otomatis dalam melakukan koreksi nutrisi dan *pH*. Selain itu, halaman ini juga menyediakan pengaturan lainnya seperti durasi waktu aktivasi pompa untuk kontrol manual serta kolom *input* untuk *Token API* yang disajikan dengan fitur *masking* atau menyembunyikan karakter, guna menjamin keamanan kredensial saat aplikasi berinteraksi dengan layanan *backend*.

File	Size	Download Size	% of Download Size	Compressed	Alignment
classes.dex	2.8 MB	2.8 MB	100%	Yes	
resources.arsc	537.6 KB	143.4 KB	4.5%	No	
res	125.4 KB	124.1 KB	3.5%		
okhttp3	38.1 KB	36.1 KB	1.1%		
META-INF	30.4 KB	32.6 KB	1%		

Class	Defined Methods	Referenced Methods	Size
androidx	26583	26624	4.3 MB
com	3440	3460	433 KB
android	47	2924	29.1 KB
kotlin	1875	1875	196.6 KB
kotlinx	1847	1848	239.1 KB
java	1556	1556	12.7 KB
org	1353	1400	126.2 KB
id	661	661	103.2 KB
okhttp3	643	646	103.8 KB
okio	458	458	56.9 KB
retrofit2	258	258	35.9 KB
dagger	138	138	23.6 KB
javax	4	50	3.6 KB
a	30	30	12.4 KB
sun	22	22	179 B
d	5	5	245 B
h	5	5	446 B
c	4	4	570 B
g	4	4	534 B
i	2	2	161 B
COROUTINE	1	1	335 B
Byte[]	1	1	8 B
char[]	1	1	8 B
int[]	1	1	8 B
byte[]	1	1	8 B

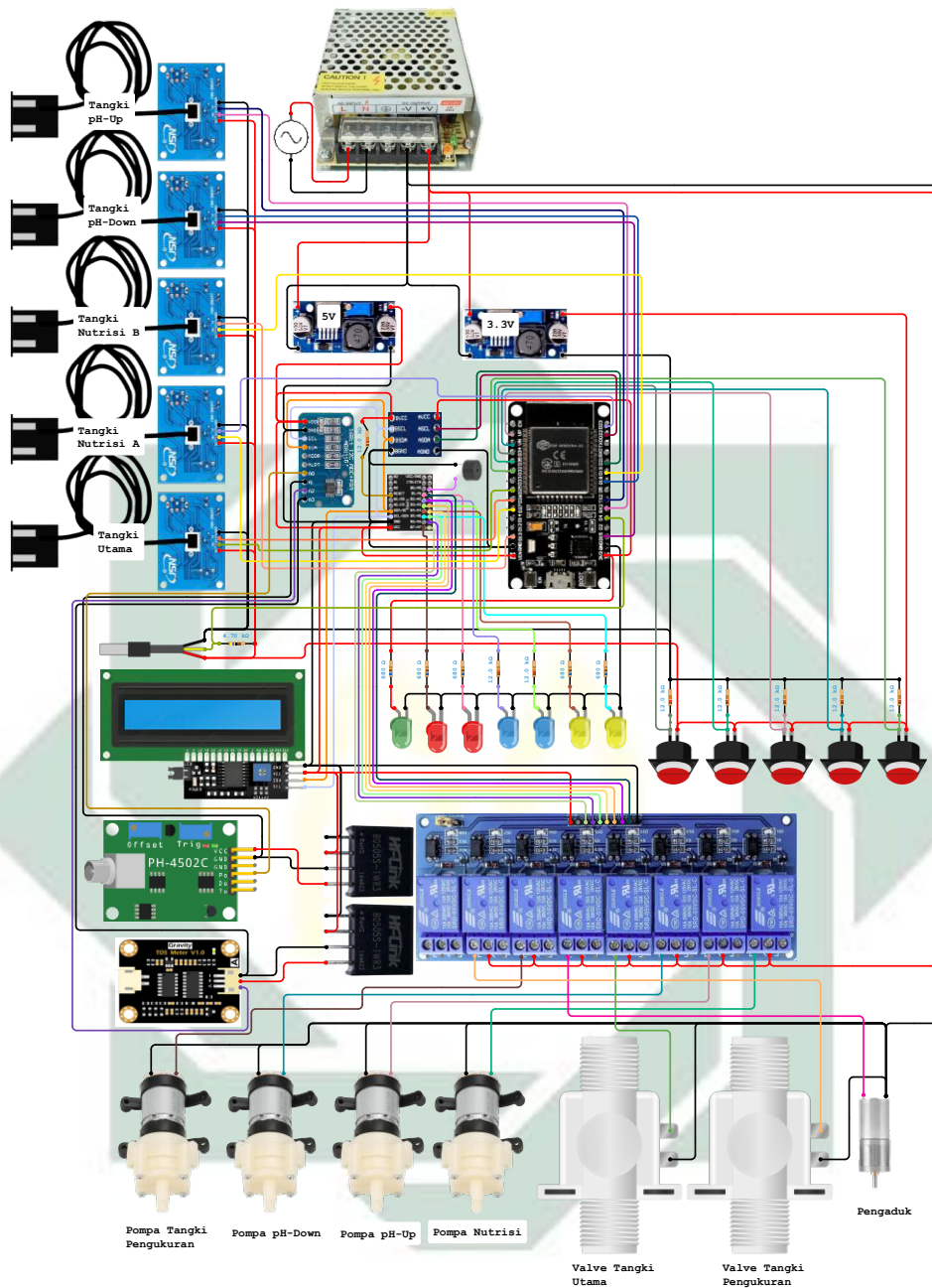
Gambar 4.31. Perbandingan saat ProGuard Aktif (Kiri) dan Tidak Aktif (Kanan)

Tahap akhir dari pengembangan aplikasi adalah proses *build* menjadi paket aplikasi siap pasang atau APK. Gambar 4.31 memperlihatkan analisis perbandingan struktur internal dan ukuran *file* antara APK yang menggunakan fitur *ProGuard* (*R8 Compiler*) dengan yang tidak. Sisi kiri gambar menunjukkan versi rilis dengan *ProGuard* aktif yang memiliki ukuran *file* jauh lebih ringkas, yaitu sekitar 3.6 MB, berbanding jauh dengan versi yang tanpa *ProGuard* di sisi kanan yang mencapai 11.6 MB. Penurunan ukuran yang signifikan ini terjadi karena *ProGuard* melakukan proses *shrinking* dengan membuang kode dan *resource* yang tidak terpakai. Lebih dari sekadar efisiensi penyimpanan, *ProGuard* juga menerapkan teknik *obfuscasi* yang mengubah nama kelas dan metode menjadi karakter acak yang sulit dimengerti, sebagaimana terlihat pada daftar struktur kelas di sisi kiri, sehingga memberikan lapisan keamanan tambahan terhadap upaya rekayasa balik (*reverse engineering*) oleh pihak yang tidak bertanggung jawab.

4.1.1.3.3. Membangun Perangkat *IoT*

Sebagai bentuk transparansi pengembangan dan guna mendukung proses replikasi pada perangkat keras, keseluruhan instruksi dan logika *program* (*source code*) yang ditanamkan pada *microcontroller* telah disimpan pada repositori daring. Repositori kode sumber untuk perangkat *Internet of Things (IoT)* ini dapat diakses secara publik melalui platform GitHub pada tautan berikut: <https://github.com/Anomali99/MyHydroponic-IoT>.

a. *Wiring Diagram*



Gambar 4.32. *Wiring Diagram*

Pada Gambar 4.32 merupakan *wiring diagram* perangkat keras, sistem elektronik yang dirancang dengan manajemen daya terpusat namun terdistribusi untuk menjamin stabilitas operasional komponen yang memiliki level tegangan berbeda. Sumber daya utama diperoleh dari *Power Supply Switching* (AC ke DC

12V) yang kemudian diregulasi ulang menggunakan dua modul *Step-Down (Buck Converter)*. Modul pertama menurunkan tegangan menjadi 5V untuk menyuplai daya bagi komponen yang membutuhkan daya lebih besar atau logika 5V, seperti modul ADC ADS1115, *I/O Expander* MCP23017, Layar LCD, Modul *Relay*, *microcontroller* ESP32. Modul kedua menurunkan tegangan menjadi 3.3V khusus untuk menyuplai sensor-sensor dan tombol interaksi yang terhubung langsung ke GPIO. Pemisahan jalur daya ini krusial untuk mencegah pembebanan berlebih pada regulator internal ESP32 dan menjaga stabilitas sinyal logika.

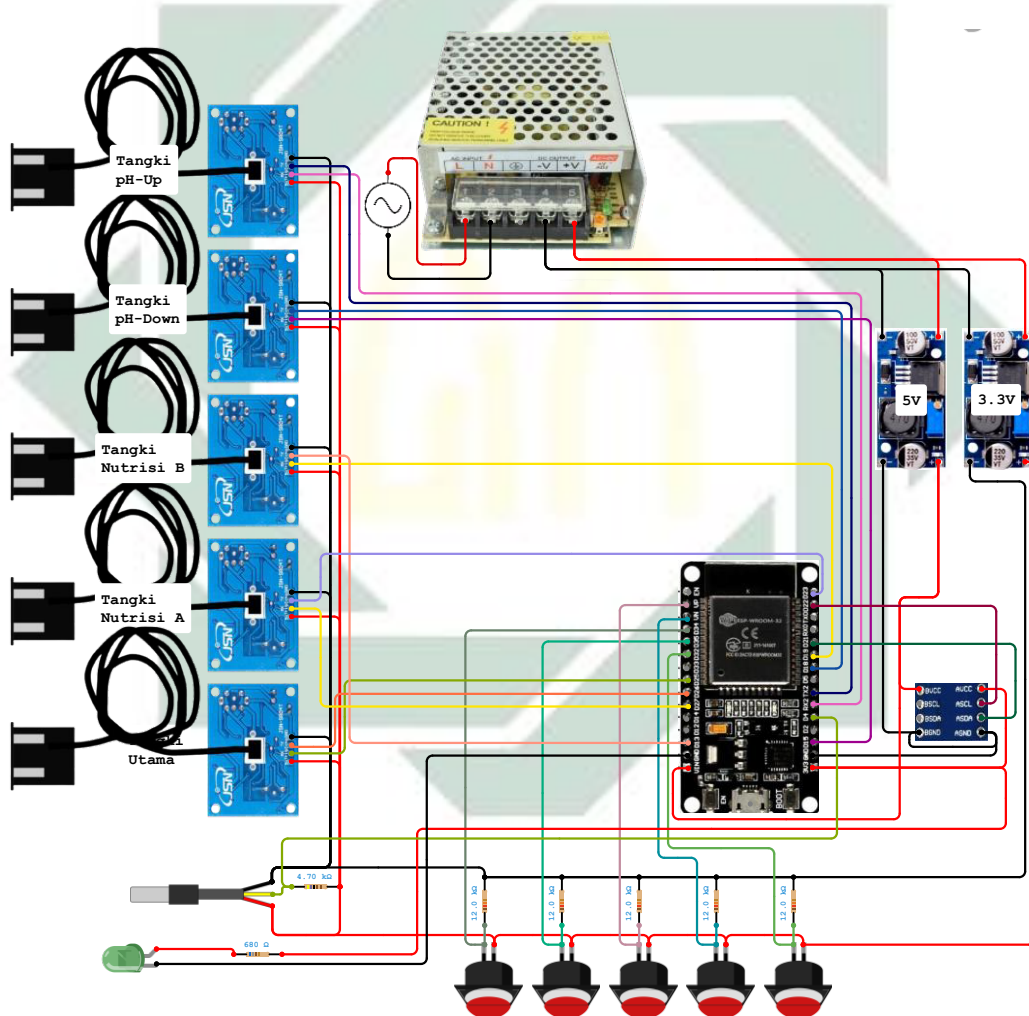
Sebagai otak utama, *Development Board* ESP32 menangani pemrosesan logika kontrol. Untuk *input* data, sistem ini memisahkan sensor ke dalam dua domain tegangan. Kelompok pertama adalah sensor dan *input* yang bekerja pada tegangan 3.3V dan terhubung langsung ke pin GPIO ESP32. Kelompok ini meliputi sensor suhu DS18B20 (GPIO 4), lima unit sensor ultrasonik (GPIO 13, 15-19, 23, 25-27) yang memantau level tangki (Utama, *pH-Up*, *pH-Down*, Nutrisi A, dan Nutrisi B), serta lima tombol interaksi fisik (GPIO 32-39) untuk kontrol manual.

Kelompok *input* kedua adalah sensor presisi yang membutuhkan tegangan referensi 5V. Sistem menggunakan modul ADC eksternal ADS1115 untuk membaca sensor analog *pH* dan TDS. Demi menjaga integritas sinyal analog, suplai daya untuk sensor *pH* dan TDS diisolasi menggunakan modul DC-DC *Converter* B0505S-1W. Modul ini berfungsi memisahkan *ground* sensor dari sirkuit utama, yang sangat efektif untuk menghilangkan *noise* akibat *ground loop* saat kedua sensor dicelupkan ke dalam air yang sama. Karena modul ADS1115, serta komponen antarmuka lain seperti Layar LCD dan *I/O Expander* MCP23017, bekerja pada tegangan 5V, komunikasi data melalui protokol I2C dijumpai oleh modul *Bi-Directional Logic Level Shifter*. Komponen ini berfungsi mengonversi sinyal logika 5V dari periferal menjadi 3.3V agar aman diterima oleh ESP32, dan sebaliknya.

Pada sisi aktuator dan indikator, sistem menggunakan *I/O Expander* (seri MCP23017) untuk memperluas jumlah pin kontrol, mengingat terbatasnya pin GPIO pada ESP32. Melalui komunikasi I2C, *expander* ini mengontrol modul *Relay* 8-Channel (7 Channel yang digunakan) dan komponen antarmuka pengguna. *Relay*

bertugas menyalakan komponen bertegangan 12V, yang terdiri dari: empat pompa mini (untuk *Measuring Tank*, *pH-Up*, *pH-Down*, dan *Nutrisi*), dua *Solenoid Valve* (untuk pengisian *Main Tank* dan pengurasan *Measuring Tank*), serta satu motor *Mixer*. Selain itu, *expander* ini juga mengontrol *Buzzer* dan lima lampu LED indikator (*Heartbeat*, *Warning*, *WiFi*, *MQTT*, *Read/Add Event*) untuk memberikan umpan balik status sistem secara visual dan audial kepada pengguna.

Untuk memastikan implementasi perangkat keras sesuai dengan desain skematis di atas, detail koneksi antar-komponen diuraikan secara spesifik. Tabel 4.3 dan Gambar 4.33 di bawah ini merinci konfigurasi pin pada *microcontroller* ESP32, menunjukkan koneksi langsung ke sensor, tombol, dan regulator daya.



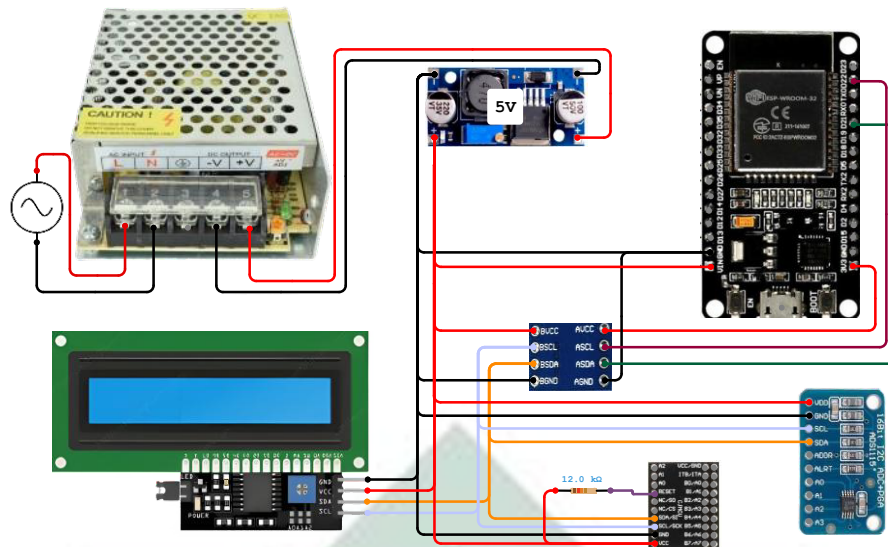
Gambar 4.33. Wiring Diagram Konfigurasi Pin ESP32

Tabel 4.3. Konfigurasi Pin ESP32

Modul	Pin Modul	Pin ESP32	LM2596 (3.3V/5V)
ESP32	VIN		OUT + (5V)
	GND		OUT – (5V)
<i>Logic Shifter</i>	AVCC	3V3	
	ASCL	D22	
	ASDA	D21	
	AGND	GND	
DS18B20	VCC		OUT + (3.3V)
	DATA	D4 (4.7k Ω dengan VCC)	
	GND		OUT – (3.3V)
JSN-SR04T (Tangki Utama)	5V		OUT + (3.3V)
	TRIG	D25	
	ECHO	D26	
	GND		OUT – (3.3V)
JSN-SR04T (Tangki Nutrisi A)	5V		OUT + (3.3V)
	TRIG	D27	
	ECHO	D23	
	GND		OUT – (3.3V)
JSN-SR04T (Tangki Nutrisi B)	5V		OUT + (3.3V)
	TRIG	D19	
	ECHO	D13	
	GND		OUT – (3.3V)
JSN-SR04T (Tangki <i>pH-Up</i>)	5V		OUT + (3.3V)
	TRIG	D15	
	ECHO	D18	
	GND		OUT – (3.3V)
JSN-SR04T (Tangki <i>pH-Down</i>)	5V		OUT + (3.3V)
	TRIG	D16	
	ECHO	D17	

Modul	Pin Modul	Pin ESP32	LM2596 (3.3V/5V)
	GND		OUT – (3.3V)
Tombol (Reconnect)	PIN1	D34	OUT – (3.3V) (12k Ω dengan D34)
	PIN2		OUT + (3.3V)
Tombol (Baca Data)	PIN1	D35	OUT – (3.3V) (12k Ω dengan D35)
	PIN2		OUT + (3.3V)
Tombol (Tambah <i>pH-Up</i>)	PIN1	VP	OUT – (3.3V) (12k Ω dengan VP)
	PIN2		OUT + (3.3V)
Tombol (Tambah <i>pH-Down</i>)	PIN1	VN	OUT – (3.3V) (12k Ω dengan VN)
	PIN2		OUT + (3.3V)
Tombol (Tambah Nutrisi)	PIN1	D32	OUT – (3.3V) (12k Ω dengan D32)
	PIN2		OUT + (3.3V)
LED Indikator Power	+	3.3V (680 Ω dengan +)	
	–	GND	

Karena beberapa modul beroperasi pada tegangan logika 5V sementara ESP32 menggunakan 3.3V, komunikasi data I2C harus melalui perantara untuk memastikan kompatibilitas dan keamanan sinyal. Tabel 4.4 dan Gambar 4.34 berikut ini menjabarkan konfigurasi sambungan pada modul *Logic Level Shifter*, yang berfungsi sebagai jembatan antara domain tegangan 3.3V dan 5V.



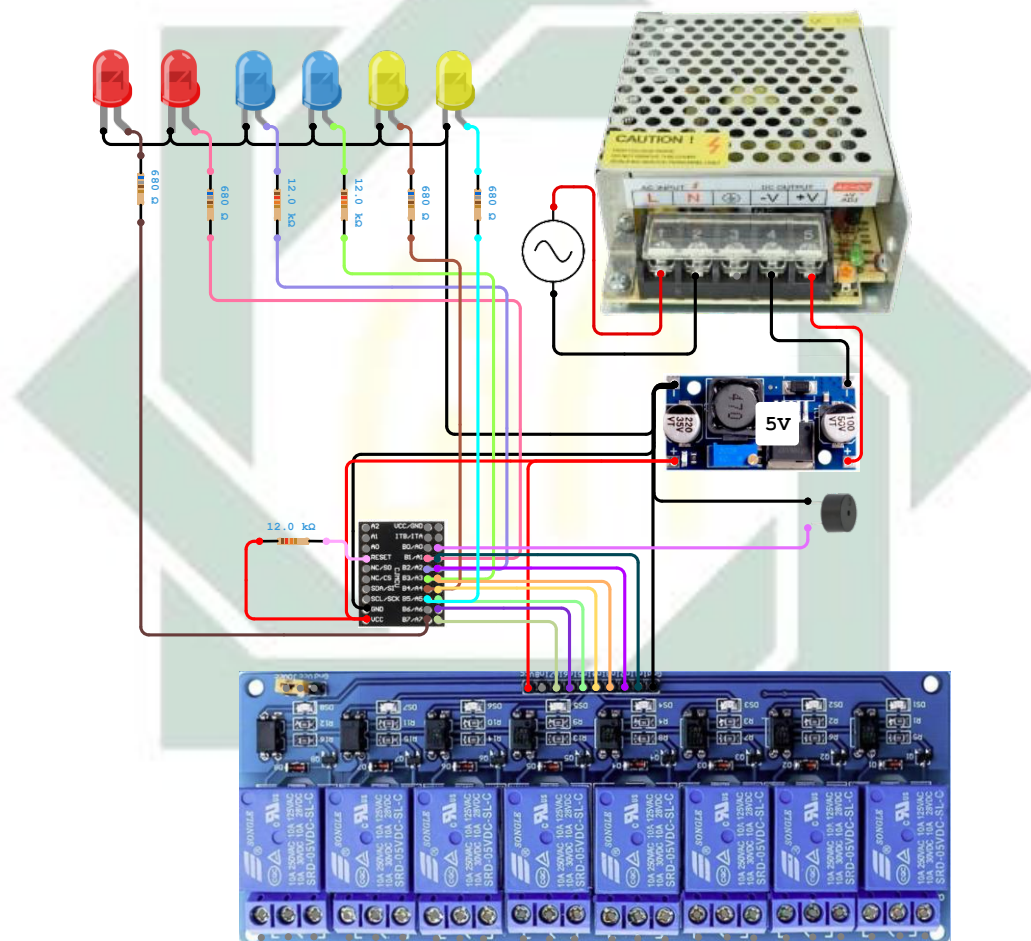
Gambar 4.34. Wiring Diagram Konfigurasi Pin Logic Shifter

Tabel 4.4. Konfigurasi Pin Logic Shifter

Modul	Pin Modul	Pin Logic Shifter	LM2596 (3.3V/5V)
ESP32	VIN	BVCC	OUT + (5V)
	GND	AGND & BGND	OUT – (5V)
	3V3	AVCC	
	D21	ASDA	
	D22	ASCL	
MCP23017	VCC		OUT + (5V)
	GND		OUT – (5V)
	SDA	BSDA	
	SCL	BSCL	
	RESET		OUT + (5V) (12k Ω dengan RESET)
ADS1115	VCC		OUT + (5V)
	GND		OUT – (5V)
	SDA	BSDA	
	SCL	BSCL	
LCD 1602	VCC		OUT + (5V)
	GND		OUT – (5V)

Modul	Pin Modul	Pin <i>Logic Shifter</i>	LM2596 (3.3V/5V)
	SDA	BSDA	
	SCL	BSCL	

Setelah komunikasi I2C dipastikan aman dan stabil, kita beralih ke manajemen aktuator dan indikator. Keterbatasan pin GPIO pada ESP32 diatasi dengan menggunakan MCP23017 *I/O Expander*. Tabel 4.5 dan Gambar 4.35 di bawah ini menguraikan bagaimana pin-pin pada *expander* ini dialokasikan untuk mengontrol modul *Relay 8-Channel*, *Buzzer*, dan serangkaian LED indikator status.



Gambar 4.35. Wiring Diagram Konfigurasi Pin MCP23017

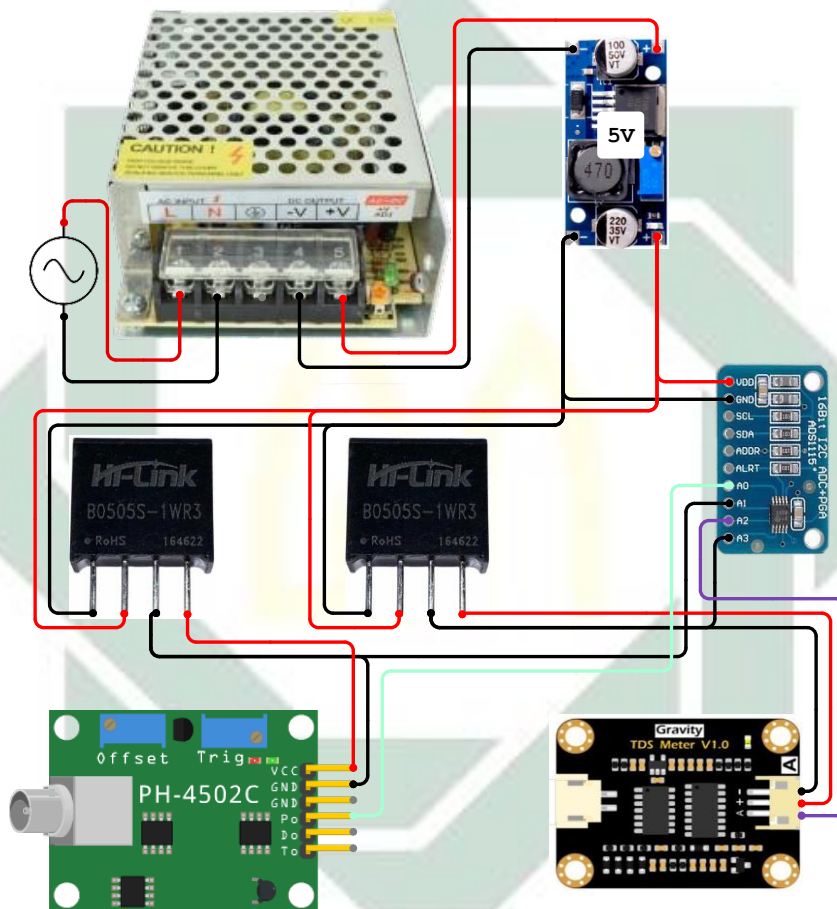
Tabel 4.5. Konfigurasi Pin MCP23017

Modul	Pin Modul	Pin MCP23017	LM2596 (3.3V/5V)
<i>Buzzer</i>	+	A0	

Modul	Pin Modul	Pin MCP23017	LM2596 (3.3V/5V)
	–		OUT – (5V)
<i>Relay 8 Channel</i>	VCC		OUT + (5V)
	GND		OUT – (5V)
	IN1	A1	
	IN2	A2	
	IN3	A3	
	IN4	A4	
	IN5	A5	
	IN6	A6	
	IN7	A7	
<i>LED Heartbeat</i>	+	B7 (680 Ω dengan +)	
	–		OUT – (5V)
<i>LED Warning</i>	+	B1 (680 Ω dengan +)	
	–		OUT – (5V)
<i>LED Status WiFi</i>	+	B2 (12k Ω dengan +)	
	–		OUT – (5V)
<i>LED Status MQTT</i>	+	B3 (12k Ω dengan +)	
	–		OUT – (5V)
<i>LED Pembacaan Data</i>	+	B4 (680 Ω dengan +)	
	–		OUT – (5V)
<i>LED Pemberian Larutan</i>	+	B5 (680 Ω dengan +)	
	–		OUT – (5V)

Langkah terakhir dalam penyiapan sensor presisi analog adalah mengamankan akurasi sinyal. Sensor *pH* dan TDS membutuhkan pembacaan

resolusi tinggi dengan tegangan referensi 5V yang ditangani oleh ADS1115 ADC Eksternal. Selain itu, sistem ini mengintegrasikan modul DC-DC *converter* B0505S-1W untuk memberikan isolasi daya sensor. Dengan memisahkan *ground* dan sumber daya sensor dari sirkuit utama *microcontroller*, modul ini memastikan sinyal analog yang dikirim ke ADS1115 tetap stabil dan bebas dari *noise*. Tabel 4.6 dan Gambar 4.36 menunjukkan detail koneksi pin sensor *pH* (PH-4502C) dan sensor TDS, termasuk jalur daya terisolasi dari B0505S-1W ke saluran *input* analog pada modul ADS1115.



Gambar 4.36. Wiring Diagram Konfigurasi Pin ADS1115

Tabel 4.6. Konfigurasi Pin ADS1115

Modul	Pin Modul	Pin ADS1115	Pin B0505S-1W	LM2596 (3.3V/5V)
PH-4502C	V+		OUT + (1)	
	G	A1	OUT – (1)	
	Po	A0		

Modul	Pin Modul	Pin ADS1115	Pin B0505S-1W	LM2596 (3.3V/5V)
			IN + (1)	OUT + (5V)
			IN – (1)	OUT – (5V)
Sensor TDS	+		OUT + (2)	
	–	A3	OUT – (2)	
	A	A2		
			IN + (2)	OUT + (5V)
			IN – (2)	OUT – (5V)

b. Desain Perangkat



Gambar 4.37. Keseluruhan Perangkat IoT

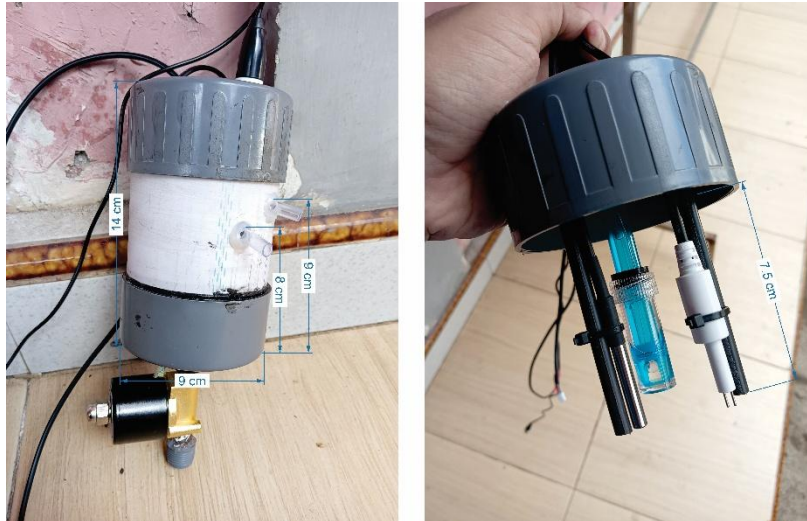
Gambar 4.37 memperlihatkan integrasi keseluruhan sistem yang telah dirakit. Sistem ini menggabungkan tempat tanaman, tangki utama, tangki koreksi, tangki pengukuran, kotak kontrol dan tandon. Sistem dirancang agar ringkas dan portabel, memungkinkan pembongkaran ketika mau dipindahkan.



Gambar 4.38. Tangki Koreksi

Tangki koreksi berfungsi sebagai wadah penyimpanan koreksi (nutrisi A, nutrisi B, *pH-Up* dan *pH-Down*) yang akan ditambahkan ke tangki utama. Seperti terlihat pada Gambar 4.38, unit ini terdiri dari empat tabung vertikal berbahan pipa PVC berdiameter 4 cm dengan ketinggian total mencapai 53 cm. Setiap tabung dilengkapi dengan selang di bagian bawah belakang yang terhubung ke pompa untuk proses injeksi nutrisi.

Untuk pemantauan volume cairan, sistem ini menggunakan dua metode: selang transparan di bagian depan sebagai indikator visual ketinggian larutan, serta sensor ultrasonik yang terpasang di bagian atas setiap tabung untuk mendeteksi ketersediaan larutan koreksi. Pada bagian atas pipa terdapat rongga sekitar 4,5 x 2,5 cm. Rongga ini bertujuan untuk mengantisipasi galat saat melakukan pembacaan sensor ultrasonik akibat gema dari dinding pipa.



Gambar 4.39. Tangki Pengukuran

Tangki pengukuran (Gambar 4.39) adalah wadah khusus yang dirancang untuk menampung sensor-sensor kualitas air, yaitu sensor *pH*, sensor TDS, dan sensor suhu air. Wadah ini dibuat dari pipa PVC berdiameter 9 cm dengan tinggi tabung sekitar 14 cm. Secara fisik, tabung ini dilengkapi dengan dua konektor selang di bagian samping; selang yang posisinya lebih tinggi berfungsi sebagai jalur masuk untuk air sampel dari tangki utama, sedangkan selang yang posisinya lebih rendah berfungsi sebagai jalur pengendali luapan (*overflow*) untuk mengembalikan kelebihan air ke tangki utama.

Desain tangki terpisah ini bertujuan menjaga keawetan sensor agar *probe* tidak terendam air secara terus-menerus (24/7). Mekanismenya, tangki pengukuran ini hanya akan diisi air ketika melakukan pengukuran, dan setelah data selesai dibaca, air sampel dibuang melalui *solenoid valve* yang terpasang di bagian bawah tabung.



Gambar 4.40. Tangki Utama

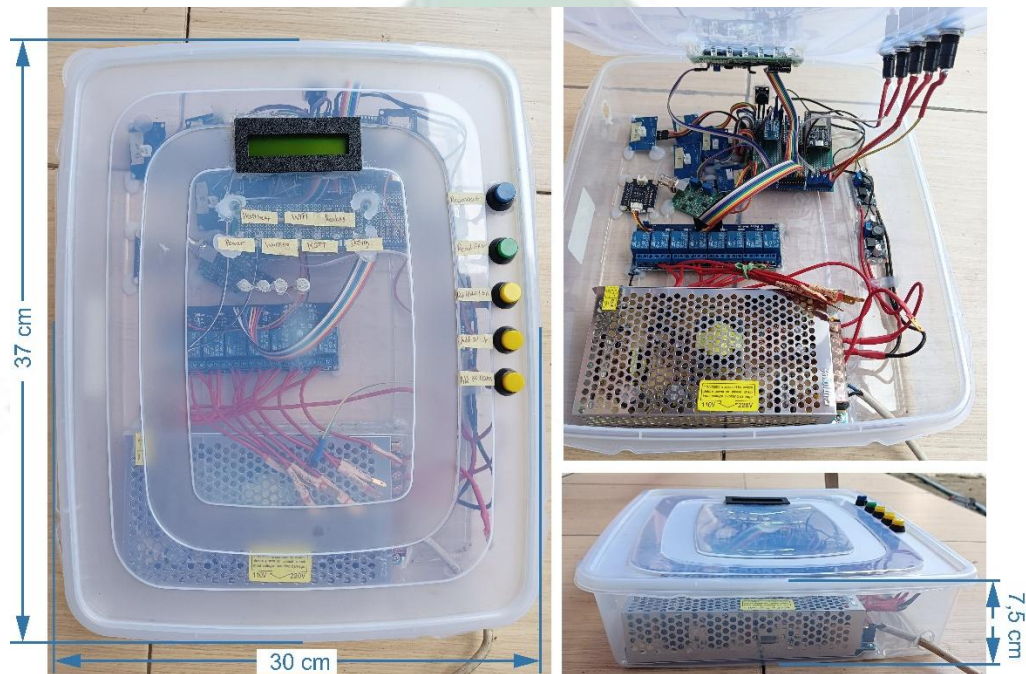
Tangki utama (Gambar 4.40) menggunakan wadah kontainer plastik dengan dimensi panjang 46 cm, lebar 34 cm, dan tinggi 30 cm. Wadah ini berfungsi sebagai penampungan utama air sirkulasi sekaligus tempat berlangsungnya proses pencampuran nutrisi. Pada bagian atas penutup tangki, terdapat kompartemen tambahan berdimensi 27 cm x 27 cm x 9 cm yang menampung empat buah pompa DC. Tiga pompa bertugas menyedot cairan dari tangki koreksi, dan satu pompa lainnya berfungsi mengalirkan air ke tangki pengukuran.

Untuk meningkatkan akurasi pembacaan level air, sistemudukan sensor ultrasonik dirancang dengan struktur khusus. Di bagian luar, sensor terpasang pada adaptor pipa berdiameter 4 cm setinggi 1,5 cm, yang didudukkan di atas pipa utama berdiameter 9 cm setinggi 12,5 cm. Kombinasi ini menempatkan posisi sensor pada ketinggian total sekitar 14 cm dari permukaan penutup. Elevasi ini sangat krusial untuk mengatasi *blind zone* sensor, sehingga hanya memerlukan jarak 6 cm dari penutup tangki untuk ketinggian maksimal air agar tetap terbaca oleh sensor.

Struktur pipa berdiameter 9 cm ini tidak hanya berada di luar, tetapi diteruskan menembus ke bagian dalam tangki sepanjang 17 cm. Perpanjangan pipa

ke dalam ini berfungsi sebagai tabung penenang yang melindungi area permukaan air di bawah sensor dari riak gelombang yang dihasilkan oleh perputaran mixer, sehingga data jarak yang dibaca sensor tetap stabil.

Di luar area tabung penenang tersebut, terdapat dua komponen aktif utama. Pertama, sebuah pompa celup yang berfungsi untuk mengalirkan air menuju instalasi tanaman. Kedua, motor pengaduk (*mixer*) dengan poros logam sepanjang 22 cm dan bilah pengaduk berdimensi 7 cm x 3,5 cm. Pengaduk ini bertugas memastikan larutan nutrisi tercampur rata sebelum pengambilan sampel maupun setelah proses koreksi.



Gambar 4.41. Kotak Kontrol

Pusat kendali sistem ditempatkan dalam boks plastik transparan berdimensi 37 cm x 30 cm x 7,5 cm untuk melindungi komponen elektronik dari percikan air (Gambar 4.41). Di dalamnya tersusun rapi komponen-komponen vital, meliputi *Power Supply Unit (PSU)* jaring untuk mengubah arus AC ke DC, modul *relay 8-channel* untuk mengendalikan aktuator (pompa dan solenoid), *microcontroller* sebagai otak pemrosesan, serta *PCB (Printed Circuit Board)* antarmuka untuk sensor ultrasonik, *pH*, dan TDS. Pada bagian luar tutup boks, terdapat antarmuka pengguna berupa layar LCD 20x4 untuk menampilkan data lingkungan yang telah dibaca dan lima buah tombol untuk kontrol manual, dan tujuh buah LED indikator.



Gambar 4.42. Sirkulasi Tanaman

Gambar 4.42 mengilustrasikan modul penanaman yang menggunakan sistem pipa PVC horizontal berlubang. Konstruksi modul ini terdiri dari dua batang pipa utama dengan panjang 52,5 cm dan diameter sekitar 11,5 cm yang ditopang oleh rangka kaki berbahan PVC. Mekanisme irigasi menerapkan sistem sirkulasi tertutup, di mana air didistribusikan dari tangki utama melalui selang *inlet* berwarna biru ke salah satu ujung pipa, kemudian mengalir kembali ke tangki dengan memanfaatkan gaya gravitasi.



Gambar 4.43. Tandon

Sebagai sumber air baku, digunakan tandon berupa galon air berkapasitas 19 liter (Gambar 4.43). Tandon ini berfungsi untuk menyediakan air bersih ketika level air di tangki utama berkurang akibat evaporasi atau transpirasi tanaman. Pada bagian mulut bawah galon, dipasang sebuah *solenoid valve*. *Valve* ini akan

membuka secara otomatis berdasarkan perintah dari *microcontroller* untuk mengisi ulang air ke tangki utama hingga mencapai level yang ditentukan.

4.1.1.4. Fase Pengujian Sistem

4.1.1.4.1. Pengujian Fungsional (*Blackbox Testing*)

Pengujian fungsional dilakukan berdasarkan skenario yang diturunkan langsung dari *Use Case Diagram*. Setiap skenario dirancang untuk memverifikasi apakah interaksi antara *User*, *Device IoT*, dan Sistem berjalan sesuai dengan alur aktivitas yang diharapkan.

Tabel 4.7 di bawah ini menyajikan rancangan skenario pengujian fungsional untuk fitur-fitur utama sistem, yang mencakup kode *Use Case* terkait, prosedur pelaksanaan tes, serta hasil yang diharapkan dari setiap skenario tersebut.

Tabel 4.7. Skenario Pengujian Fungsional

Kode Pengujian	Skenario Pengujian	Kode Use Case	Prosedur Pengujian	Hasil yang Diharapkan
BB01	Mengatur Ambang Batas Kontrol Otomatis	UC02	1. Buka menu <i>Settings</i> di aplikasi. 2. <i>Input</i> nilai <i>min/max pH</i> dan TDS baru. 3. Tekan tombol simpan.	Sistem N8N mengecek duplikasi data. Jika data baru, status <i>is_active</i> diperbarui menjadi <i>true</i> dan notifikasi sukses muncul di aplikasi.
BB02	Eksekusi Kontrol Otomatis	UC01, UC11	1. Pastikan ambang batas aktif. 2. Berikan larutan pada air hingga nilai <i>pH</i> /TDS keluar dari rentang ideal. 3. Tunggu siklus <i>trigger</i> 30 menit.	N8N mendeteksi anomali, menghitung durasi pompa, dan mengirim perintah ke topik " <i>pump/automation</i> ". <i>Device IoT</i> menyalakan pompa koreksi secara otomatis.

Kode Pengujian	Skenario Pengujian	Kode Use Case	Prosedur Pengujian	Hasil yang Diharapkan
BB03	Pemantauan Data via Aplikasi	UC03	1. <i>Buka Dashboard Monitoring.</i> 2. Bandingkan angka di aplikasi dengan kondisi aktual sensor.	Aplikasi menampilkan data (<i>pH</i> , TDS, Suhu, dan volume tangki) yang diambil dari topik " <i>environment/data</i> ".
BB04	Kontrol Manual via Aplikasi	UC04, UC11	1. Tekan tombol pompa nutrisi atau <i>pH</i> pada aplikasi. 2. Amati respons perangkat.	Perintah terkirim ke topik " <i>pump/manually</i> ". <i>Device IoT</i> segera menyalakan pompa fisik dan N8N mencatat riwayat aktivitas ke <i>database</i> .
BB05	Kontrol Manual via Lokal (Fisik)	UC06, UC11	1. Tekan tombol fisik "<i>Add pH-Up</i>" pada boks perangkat. 2. Amati indikator LED dan pompa.	Sistem memvalidasi status kesibukan. Jika siap, pompa menyala, LED indikator menyala, dan tangki diaduk otomatis setelah pemberian larutan.
BB06	Notifikasi Volume Larutan Rendah	UC07, UC09	1. Kosongkan tangki larutan koreksi hingga di bawah sensor ultrasonik. 2. Tunggu siklus pembacaan.	<i>Device IoT</i> membunyikan <i>buzzer</i> dan mengirim pesan ke topik " <i>tank/notification</i> " yang diteruskan menjadi <i>push notification</i> ke pengguna.

Kode Pengujian	Skenario Pengujian	Kode Use Case	Prosedur Pengujian	Hasil yang Diharapkan
BB07	Notifikasi Perangkat <i>Offline</i>	UC08	1. Putuskan daya atau koneksi internet pada <i>Device IoT</i> . 2. Tunggu respons sistem.	N8N mendeteksi status perangkat tidak tersedia melalui topik " <i>device/status</i> " dan mengirimkan <i>push notification</i> ke pengguna.
BB08	Mengambil & Menampilkan Data Lokal	UC05, UC10	1. Tekan tombol "Baca Data" pada perangkat. 2. Lihat layar LCD.	Sensor membaca data lingkungan dan hasilnya langsung ditampilkan pada layar LCD lokal.

Setelah rancangan skenario disusun, pengujian dilaksanakan menggunakan perangkat prototipe dan aplikasi. Tabel 4.8 berikut merangkum hasil observasi dari pengujian yang telah dilakukan, membandingkan hasil aktual sistem dengan hasil yang diharapkan untuk menentukan validitas fungsi.

Tabel 4.8. Hasil Pengujian Fungsional

Kode Pengujian	Skenario Pengujian	Hasil Aktual	Status
BB01	Mengatur Ambang Batas Kontrol Otomatis	Pada aplikasi, <i>input</i> nilai ambang batas berfungsi dengan baik dan tombol simpan merespons sentuhan. Setelah ditekan, aplikasi menampilkan notifikasi sukses. Secara bersamaan, <i>workflow</i> N8N tereksekusi, memvalidasi data <i>input</i> , dan menyimpannya ke <i>database limit_values</i> dengan benar.	Valid

Kode Pengujian	Skenario Pengujian	Hasil Aktual	Status
BB02	Eksekusi Kontrol Otomatis	Saat data sensor yang diterima berada di luar ambang batas ideal, <i>workflow</i> N8N secara otomatis mengarahkan logika ke jalur koreksi yang sesuai melalui <i>node Switch</i> . Sistem sukses menghitung durasi pompa yang dibutuhkan, mengirimkan perintah aktivasi pompa ke perangkat <i>IoT</i> , dan mencatat tindakan koreksi tersebut ke <i>database</i> tanpa intervensi manual pengguna.	Valid
BB03	Pemantauan Data via Aplikasi	Halaman <i>Dashboard Monitoring</i> berhasil menampilkan data terbaru. Parameter utama (<i>pH</i> , TDS, Suhu) dan status volume air tangki nilai sesuai dengan pembacaan sensor pada <i>Device IoT</i> , dengan sinkronisasi data yang berjalan lancar melalui protokol MQTT.	Valid
BB04	Kontrol Manual via Aplikasi	Saat tombol ditekan pada aplikasi, <i>workflow</i> N8N merespons instan dan meneruskan perintah ke perangkat <i>IoT</i> . Secara fisik, pompa berhasil menyala untuk menambahkan larutan yang sesuai target, dan sistem sukses mencatat riwayat aktivitas tersebut ke <i>database</i> sebagai konfirmasi eksekusi.	Valid
BB05	Kontrol Manual via Lokal (Fisik)	Saat tombol fisik ditekan, sinyal diterima oleh <i>workflow</i> N8N. Sistem berhasil memproses permintaan tersebut, menyalakan pompa dan LED indikator pada perangkat, serta	Valid

Kode Pengujian	Skenario Pengujian	Hasil Aktual	Status
		mencatat aktivitas manual ini ke <i>database</i> dengan sukses.	
BB06	Notifikasi Volume Larutan Rendah	Saat volume air terdeteksi di bawah batas, <i>workflow</i> N8N menerima <i>trigger</i> dan segera memproses logika peringatan. Sistem berhasil mengirimkan notifikasi <i>push</i> ke HP pengguna yang berisi informasi spesifik mengenai tangki yang hampir habis beserta sisa volume aktualnya.	Valid
BB07	Notifikasi Perangkat <i>Offline</i>	Setelah daya perangkat diputus, <i>workflow</i> pada N8N berhasil mendeteksi hilangnya koneksi. Sistem segera mengirimkan notifikasi peringatan ke aplikasi pengguna, dan status indikator konektivitas pada <i>dashboard</i> berubah menjadi “ <i>Offline</i> ”.	Valid
BB08	Mengambil & Menampilkan Data Lokal	Saat tombol fisik “ <i>Read Env</i> ” ditekan, sistem segera memulai proses pengambilan data sensor terbaru. Hasil pembacaan kemudian ditampilkan pada layar LCD secara bergantian (<i>scrolling</i>), meliputi informasi volume cairan, nilai <i>pH</i> , TDS, suhu serta waktu pengambilan data, yang memvalidasi bahwa fungsi baca data lokal berjalan sukses.	Valid

4.1.1.4.2. Pengujian Kompatibilitas

Mengingat kebutuhan non-fungsional aspek “Kemudahan Penggunaan”, sistem diwajibkan untuk menjamin kompatibilitas perangkat yang optimal agar dapat diakses oleh pengguna dengan berbagai spesifikasi perangkat. Pengujian ini dilakukan dengan memasang dan menjalankan aplikasi pada serangkaian perangkat uji yang memiliki variasi versi sistem operasi Android. Tujuannya adalah untuk memastikan bahwa fungsi-fungsi vital berjalan lancar tanpa mengalami kendala teknis (*crash* atau *force close*) pada lintas generasi sistem operasi.

Hasil pengujian kompatibilitas pada berbagai versi Android dapat dilihat pada Tabel 4.9.

Tabel 4.9. Hasil Pengujian Kompatibilitas

Versi Android	Hasil Tampilan	Hasil Fungsionalitas	Kesimpulan
Android 6	Tidak dapat diobservasi.	Gagal instalasi. Muncul pesan <i>error</i> sistem "Terjadi masalah saat mengurai paket".	Tidak Kompatibel
Android 7	Tidak dapat diobservasi.	Gagal instalasi. Muncul pesan <i>error</i> sistem "Terjadi masalah saat mengurai paket".	Tidak Kompatibel
Android 10	Tampilan antarmuka <i>me-render</i> dengan baik, posisi <i>card</i> sensor dan tombol navigasi tertata rapi tanpa elemen yang terpotong.	Fitur dasar seperti navigasi halaman dan <i>refresh</i> data berjalan normal tanpa <i>force close</i> .	Kompatibel
Android 11	Tata letak elemen konsisten. Mode Terang (<i>Light Mode</i>) menampilkan warna hijau dan teks yang kontras serta mudah dibaca.	<i>Input</i> numerik pada pengaturan ambang batas berfungsi, <i>keyboard</i> muncul dengan responsif, dan tombol simpan bekerja.	Kompatibel
Android 13	UI responsif pada orientasi <i>portrait</i> . Tidak terjadi	Validasi <i>input</i> berjalan baik (muncul pesan <i>Toast</i>	Kompatibel

Versi Android	Hasil Tampilan	Hasil Fungsionalitas	Kesimpulan
	<i>overlapping</i> teks pada label sensor maupun status tangki cairan.	“Token API ditolak” saat token salah), serta <i>toggle switch</i> notifikasi berfungsi lancar.	
Android 14	Transisi antar halaman halus. Ikon dan <i>progress bar</i> pada level tangki di-render dengan sempurna.	Sistem notifikasi <i>overlay</i> berjalan baik di atas aplikasi, dan fungsionalitas tombol "Batal/Simpan" merespons sentuhan tanpa <i>delay</i> .	Kompatibel
Android 15	Implementasi Mode Gelap (<i>Dark Mode</i>) sangat baik; warna latar belakang dan teks kontras, elemen <i>input</i> pada pengaturan terlihat jelas.	Seluruh kolom <i>input</i> (<i>pH</i> , TDS, Durasi Pompa) dapat diakses dan diedit. Tombol <i>switch</i> otomatisasi merespons interaksi dengan benar.	Kompatibel

4.1.1.4.3. Pengujian Keandalan Sistem

Pengujian keandalan difokuskan pada kemampuan sistem untuk beroperasi terus-menerus dan pulih dari kegagalan, sesuai dengan kebutuhan non-fungsional. Aspek yang diuji meliputi kemampuan *auto-reconnect* saat terjadi gangguan daya, toleransi terhadap putusnya jaringan internet, serta ketahanan fisik perangkat terhadap lingkungan lembap hidroponik.

Tabel 4.10 merangkum skenario dan hasil uji keandalan sistem.

Tabel 4.10. Hasil Pengujian Keandalan Sistem

Aspek Keandalan	Skenario Gangguan	Mekanisme Pemulihan Sistem	Hasil Pengujian
Pemulihan Kegagalan	Mematikan paksa catu daya <i>Device IoT</i> saat sedang	Perangkat melakukan setup ulang dan menghubungkan kembali ke WiFi serta MQTT	Berhasil. Setelah pemulihan daya, perangkat melakukan <i>reboot</i> dan memasuki

Aspek Keandalan	Skenario Gangguan	Mekanisme Pemulihan Sistem	Hasil Pengujian
	beroperasi, lalu menyalakannya kembali.	Broker secara otomatis tanpa intervensi manual.	mode setup selama 1 menit. Setelah fase setup selesai, sistem secara otomatis memulai pembacaan sensor, mengirimkan data ke MQTT Broker, serta menampilkannya pada layar LCD tanpa memerlukan intervensi manual.
Toleransi Jaringan	Memutus koneksi internet saat perangkat sedang mengirim data sensor.	Perangkat menyimpan data sementara. Saat data gagal terkirim, sistem mengalihkan data ke topik <i>"history/pending"</i> untuk diproses ulang oleh N8N sehingga data tidak hilang.	Berhasil. Log eksekusi N8N menunjukkan sistem sukses menerima dan memproses data. Hal ini memvalidasi bahwa data yang tertahan saat jaringan terputus berhasil dikirimkan kembali begitu koneksi pulih, dan tersimpan ke <i>database</i> .
Ketahanan Fisik	Memberikan percikan air pada wadah perangkat saat sedang beroperasi.	Desain tertutup mencegah percikan air masuk ke komponen elektronik, memungkinkan perangkat tetap beroperasi di lingkungan lembap.	Berhasil. Tidak ditemukan rembesan air pada bagian dalam kotak kontrol setelah pengujian. Komponen elektronik tetap kering dan perangkat beroperasi normal tanpa gangguan elektrik atau korsleting.

4.1.1.4.4. Requirement Traceability Matrix

Untuk memastikan konsistensi dan kelengkapan pengembangan sistem, disusunlah *Requirement Traceability Matrix (RTM)*. Matriks ini berfungsi untuk memetakan hubungan antara kebutuhan pengguna yang didefinisikan dalam *Use Case Diagram*, rancangan detail dalam *Activity* dan *Sequence Diagram*, hingga verifikasi akhir melalui skenario pengujian.

Tujuan dari penyusunan matriks ini adalah untuk membuktikan bahwa setiap fitur yang dirancang telah memiliki alur logika yang jelas dan telah melalui tahap pengujian yang valid, sehingga tidak ada kebutuhan sistem yang terlewat. Pemetaan penelusuran kebutuhan sistem dapat dilihat pada Tabel 4.11.

No	Kode <i>Use Case</i>	Nama <i>Use Case</i>	Kode <i>Activity Diagram</i>	Kode <i>Sequence Diagram</i>	Kode Skenario Pengujian	Status Pengujian
1	UC01	Kontrol Otomatis	AC01	SQ01	BB02	valid
2	UC02	Mengatur Ambang Batas Kontrol Otomatis	AC04	SQ04	BB01	valid
3	UC03	Memantau Data Lingkungan (Aplikasi)	AC01, AC02, AC05	SQ01, SQ02, SQ05	BB03	valid
4	UC04	Mengontrol Manual (Aplikasi)	AC02	SQ02	BB04	valid
5	UC05	Memantau Data Lingkungan (Lokal)	AC03	SQ03	BB08	valid

No	Kode Use Case	Nama Use Case	Kode Activity Diagram	Kode Sequence Diagram	Kode Skenario Pengujian	Status Pengujian
6	UC06	Mengontrol Manual (Lokal)	AC03	SQ03	BB05	valid
7	UC07	Mengirim Notifikasi Volume Larutan Rendah	AC05	SQ05	BB06	valid
8	UC08	Mengirim Notifikasi Perangkat Offline	AC05	SQ05	BB07	valid
9	UC09	Peringatan Volume Larutan Rendah (Lokal)	AC06	SQ06	BB06	valid
10	UC10	Mengambil Data Lingkungan	AC01, AC02, AC03, AC05, AC06	SQ01, SQ02, SQ03, SQ05, SQ06	BB08	valid
11	UC11	Mengontrol pH & Nutrisi	AC01, AC02, AC03	SQ01, SQ02, SQ03	BB02, BB04, BB05	valid

Berdasarkan Tabel 4.11 di atas, dapat disimpulkan bahwa seluruh fungsionalitas sistem mulai dari UC01 hingga UC11 telah teruji dan memiliki keterlacakan dokumen perancangan yang lengkap. Hal ini menunjukkan bahwa sistem telah memenuhi spesifikasi kebutuhan yang ditetapkan pada tahap analisis.

4.1.1.5. Fase Pemeliharaan

4.1.1.5.1. Prosedur Pemeliharaan Perangkat Keras

Pemeliharaan perangkat keras difokuskan pada upaya untuk menjaga kondisi fisik sensor, khususnya pada bagian *probe* atau elektroda yang bersentuhan langsung dengan air. Sensor TDS dan sensor *pH* rentan terhadap gangguan eksternal seperti pertumbuhan lumut, bio-film, atau endapan garam mineral yang dapat menutupi permukaan elektroda. Oleh karena itu, prosedur pembersihan fisik perlu dijadwalkan secara rutin untuk menjaga sensitivitas sensor.

Langkah-langkah pemeliharaan fisik sensor adalah sebagai berikut:

1. Matikan aliran listrik ke sistem *microcontroller* untuk mencegah korsleting atau kesalahan pembacaan data.
2. Angkat *probe* sensor *pH* dan TDS dari tangki pengukuran dengan melepas penutup tangki secara perlahan.
3. Bilas ujung *probe* menggunakan air bersih untuk meluruhkan sisa endapan garam mineral.
4. Bersihkan elektroda sensor *pH* menggunakan sikat gigi berbulu halus atau tisu lembut secara perlahan untuk menghilangkan lumut atau endapan, hindari gesekan kasar yang dapat menggores kaca sensor.
5. Keringkan bagian luar sensor dengan kain lap bersih sebelum dipasang kembali ke dalam sistem.

4.1.1.5.2. Implementasi Kalibrasi Berkala

Kalibrasi berkala merupakan prosedur teknis untuk menyesuaikan kembali parameter komputasi dalam perangkat lunak agar sesuai dengan kondisi fisik sensor. Seiring berjalannya waktu, sensor (terutama sensor *pH*) akan mengalami penurunan performa yang menyebabkan pembacaan tidak akurat. Oleh karena itu, kalibrasi ulang diperlukan untuk memperbarui nilai referensi tegangan pada titik asam dan netral guna mempertahankan presisi pengukuran.

Prosedur kalibrasi sensor *pH* dilakukan dengan langkah-langkah berikut:

1. Siapkan dua jenis larutan *buffer* standar, yaitu larutan dengan *pH* 4.01 (asam) dan *pH* 6.86 (netral).

2. Celupkan sensor *pH* ke dalam larutan *buffer pH* 6.86, biarkan pembacaan stabil, lalu catat nilai tegangan yang terbaca oleh sistem.
3. Bilas sensor dengan air bersih dan keringkan, kemudian celupkan ke dalam larutan *buffer pH* 4.01 dan catat kembali tegangan stabilnya.
4. Perbarui nilai variabel konstanta tegangan (*ph7Voltage* dan *ph4Voltage*) di dalam kode program berdasarkan hasil pengukuran terbaru.
5. Unggah ulang (*re-upload*) program ke *microcontroller* untuk menerapkan parameter kalibrasi yang baru.

4.1.2. Menguji Coba Sistem

Tahap uji coba sistem dilakukan selama durasi 3 (tiga) hari penuh (72 jam). Periode pengujian ini dimulai sejak tahap inisialisasi alat pada hari Jumat, 26 Desember 2025 pukul 10.24 WIB hingga pengambilan data terakhir pada hari Senin, 29 Desember 2025.

4.1.2.1. Hasil Pengujian

a. Data Perbandingan Pengukuran Manual dan *Device IoT*

Pengukuran manual menggunakan alat genggam (*handheld meter*) untuk parameter *pH* dan TDS dilakukan secara berkala dengan interval estimasi setiap 6 jam selama periode pengujian. Pengambilan data manual ini bertujuan untuk mendapatkan nilai acuan (*ground truth*) guna memvalidasi akurasi pembacaan sensor pada *Device IoT*. Tabel 4.11 berikut menyajikan perbandingan data antara hasil pengukuran manual (*y*) dan data yang terekam oleh *Device IoT* (*ŷ*) pada waktu yang bersamaan, serta nilai selisih dari kedua pengukuran tersebut.

Tabel 4.11. Perbandingan Nilai Pengukuran Manual dan *Device IoT*

Waktu Pengambilan	Manual		<i>Device IoT</i>		Selisih	
	<i>pH</i>	TDS	<i>pH</i>	TDS	<i>pH</i>	TDS
26/12/2025 10:24:43	5,81	508	5,85	503	0,04	5
26/12/2025 10:34:51	5,9	771	5,75	757	0,15	14
26/12/2025 16:00:57	4,9	885	5,94	887	1,04	2

Waktu Pengambilan	Manual		Device IoT		Selisih	
	pH	TDS	pH	TDS	pH	TDS
26/12/2025 16:23:07	4,83	885	5,98	896	1,15	11
26/12/2025 22:00:57	6,45	1060	5,06	1074	1,39	14
27/12/2025 4:00:57	7,08	1120	5,74	1105	1,34	15
27/12/2025 10:00:56	7,05	1120	6,5	1128	0,55	8
27/12/2025 12:25:21	5,8	1120	5,9	1120	0,1	0
27/12/2025 12:42:52	5,96	771	5,9	762	0,06	9
27/12/2025 16:00:57	5,36	842	6,02	842	0,66	0
27/12/2025 22:16:45	6,36	1010	5,53	1023	0,83	13
28/12/2025 4:00:57	6,9	1120	5,25	1131	1,65	11
28/12/2025 10:00:57	6,31	1120	6,94	1134	0,63	14
28/12/2025 15:44:07	4,44	1230	5,75	1226	1,31	4
28/12/2025 22:00:52	5,2	1230	5,23	1229	0,03	1
29/12/2025 4:04:11	6,09	1360	5,76	1368	0,33	8
29/12/2025 10:10:21	5,36	1360	6,39	1363	1,03	3

b. Data Respons Kontrol Otomatis

Selama periode pengujian 3 (tiga) hari, *Device IoT* merekam total 393 baris data. Mengingat volume data yang terlalu besar untuk ditampilkan secara

keseluruhan, penyajian data pada laporan ini diringkas dalam bentuk cuplikan kejadian. Tabel-tabel berikut menampilkan data dengan durasi sekitar 30 menit pada momen ketika sistem melakukan koreksi otomatis terhadap parameter Nutrisi (TDS) dan *pH*.

Tabel 4.12. Cuplikan Koreksi Nutrisi

Waktu Pengambilan	TDS (ppm)	Suhu (°C)	Volume Main Tank (ml)	Pompa Aktif	Durasi
26/12/2025 10:24:43	503	28,28	11165	NUTRITION	1,87
26/12/2025 10:31:25	497	28,40	11242	NUTRITION	1,92
26/12/2025 10:34:51	757	28,33	11242	NONE	0

Tabel 4.12 memperlihatkan respons sistem saat awal inisialisasi. Pada data tersebut, sensor TDS mendeteksi nilai konsentrasi nutrisi berada di angka 503 ppm dan 497 ppm, yang mana nilai ini masih di bawah ambang batas minimum yang ditetapkan (640 ppm). Merespons kondisi ini, sistem secara otomatis mengaktifkan pompa nutrisi dengan durasi aktif sekitar 1,8 hingga 1,9 detik untuk menambahkan larutan nutrisi ke dalam tangki utama. Setelah proses penambahan terjadi, terlihat pada pukul 10:34:51 nilai TDS berhasil naik menjadi 757 ppm, sehingga sistem tidak mengaktifkan pompa karena target telah tercapai.

Tabel 4.13. Cuplikan Koreksi pH-Up

Waktu Pengambilan	<i>pH</i>	Suhu (°C)	Volume Main Tank (ml)	Pompa Aktif	Durasi
26/12/2025 15:00:56	5,51	31,36	11974	NONE	0
26/12/2025 15:30:56	5,43	31,18	11974	PH UP	0,3

Waktu Pengambilan	<i>pH</i>	Suhu (°C)	Volume Main Tank (ml)	Pompa Aktif	Durasi
26/12/2025 15:34:20	6	31,19	11974	NONE	0

Tabel 4.13 menampilkan kejadian penambahan *pH-Up*. Berdasarkan data cuplikan di atas, sistem memantau kondisi *pH* yang awalnya stabil di angka 5,51 pada pukul 15:00:56. Namun, pada pukul 15:30:56, sensor mendeteksi penurunan nilai *pH* menjadi 5,43. Karena nilai ini berada di bawah batas minimum (5,5), sistem segera mengaktifkan pompa *pH-Up* dengan durasi 0,3 detik. Koreksi ini berhasil menaikkan nilai *pH*, di mana pada pengukuran berikutnya pukul 15:34:20, nilai *pH* telah kembali naik ke angka aman yaitu 6, dan pompa kembali ke status non-aktif.

Tabel 4.14. Cuplikan Koreksi *pH-Down*

Waktu Pengambilan	<i>pH</i>	Suhu (°C)	Volume Main Tank (ml)	Pompa Aktif	Durasi
26/12/2025 12:34:20	5,96	30,35	11916	NONE	0
26/12/2025 13:00:56	6,4	30,86	11916	PH DOWN	0,3
26/12/2025 13:04:21	5,83	30,67	11916	NONE	0

Selanjutnya, Tabel 4.14 menampilkan kejadian koreksi *pH-Down* yang terjadi pada siang hari tanggal 26 Desember. Pada pukul 13:00:56, sistem mendeteksi lonjakan nilai *pH* hingga mencapai 6,4, yang melebihi ambang batas atas (6,0). Sebagai respons korektif, sistem mengaktifkan pompa *pH-Down* untuk menyuntikkan larutan asam. Hasil dari tindakan otomatis ini terlihat pada pukul 13:04:21, di mana nilai *pH* berhasil diturunkan kembali ke level normal di angka 5,83, sehingga menjaga lingkungan perakaran tanaman tetap optimal.

4.1.2.2. Margin Error

Berdasarkan data perbandingan yang telah disajikan pada Tabel 4.11, dilakukan evaluasi akurasi sensor untuk mengetahui rata-rata penyimpangan pembacaan sistem terhadap alat ukur standar. Metode yang digunakan adalah *Mean Absolute Error (MAE)* dengan menerapkan Rumus 3.6 yang telah dijelaskan pada Bab 3. Nilai *MAE* merepresentasikan rata-rata selisih absolut antara nilai aktual (manual) dan nilai prediksi (*Device IoT*).

Perhitungan MAE_{pH} untuk menghitung tingkat kesalahan pada sensor *pH*, seluruh selisih absolut dari 17 sampel data dijumlahkan kemudian dibagi dengan banyaknya jumlah sampel ($n = 17$). Berdasarkan data pada tabel, total selisih pembacaan *pH* adalah sebesar 12,29. Maka perhitungan MAE adalah sebagai berikut:

$$MAE_{pH} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

$$MAE_{pH} = \frac{12,29}{17}$$

$$MAE_{pH} = 0,7229411765$$

Hasil perhitungan menunjukkan nilai MAE untuk parameter *pH* adalah sebesar 0,7229411765. Angka ini mengindikasikan bahwa rata-rata penyimpangan pembacaan sensor *pH* terhadap alat ukur manual adalah kurang dari 1,0 poin skala *pH*.

Perhitungan MAE_{TDS} serupa dengan parameter *pH*, perhitungan galat untuk sensor TDS dilakukan dengan menjumlahkan seluruh selisih absolut pengukuran. Total selisih pembacaan TDS dari 17 sampel data tercatat sebesar 133 ppm. Berikut adalah perhitungannya:

$$MAE_{TDS} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

$$MAE_{TDS} = \frac{133}{17}$$

$$MAE_{TDS} = 7,823529412$$

Diperoleh nilai MAE untuk parameter TDS sebesar 7,823529412 ppm. Angka ini mengindikasikan bahwa rata-rata penyimpangan pembacaan sensor TDS terhadap alat ukur manual adalah sangat rendah, yaitu kurang dari 10 ppm.

4.1.3. Menghasilkan Prototipe

Tahap menghasilkan prototipe merupakan muara akhir dari seluruh rangkaian proses penelitian dan pengembangan *R&D* pada penelitian ini. Tahap ini dicapai setelah perangkat keras dan perangkat lunak berhasil diintegrasikan dan dinyatakan lolos validasi melalui uji coba sistem selama 3 (tiga) hari (26–29 Desember 2025). Hasil akhir dari penelitian ini adalah sebuah unit prototipe fungsional skala kecil yang siap digunakan untuk otomatisasi budidaya hidroponik.

Secara fisik, prototipe dirancang agar mampu beroperasi di lingkungan hidroponik yang cenderung lembap dan basah. Ketahanan sistem dijamin dengan menempatkan seluruh komponen elektronik sensitif yang rawan korsleting—seperti *microcontroller*, modul daya, dan *relay*—ke dalam wadah panel tertutup. Dengan desain ini, komponen penting terlindungi dari cipratan air, sementara komponen *probe* dan aktuator ditempatkan secara terpisah di luar wadah. Pemisahan ini bertujuan untuk mencegah risiko hubungan arus pendek (korsleting) pada modul kontrol utama, sekaligus memungkinkan komponen tersebut berinteraksi langsung dengan larutan masing-masing.

Sistem elektronik menerapkan manajemen daya terdistribusi (12V, 5V, dan 3.3V) untuk menyuplai berbagai komponen sesuai spesifikasi tegangannya. Guna menjamin akurasi data, pembacaan sensor analog (*pH* dan TDS) diproses melalui modul ADC ADS1115 yang dilengkapi isolator tegangan B0505S-1W untuk mengurangi gangguan *noise* akibat *ground loop*. Selain itu, keterbatasan pin pada ESP32 dalam mengendalikan banyak aktuator (pompa, *valve*, *mixer*) dan indikator LED diatasi dengan penggunaan *I/O Expander* MCP23017 berbasis protokol I2C.

Dari sisi fungsionalitas, prototipe yang dihasilkan telah memiliki kemampuan untuk beroperasi secara mandiri. Sistem mampu membaca parameter lingkungan secara *real-time*, mengirimkan data ke *database* MariaDB, dan mengeksekusi

logika kontrol. Berdasarkan hasil pengujian yang telah dipaparkan sebelumnya, prototipe terbukti mampu menjaga kondisi larutan nutrisi dengan melakukan koreksi otomatis ketika nilai parameter menyimpang dari rentang ideal (pH 5.5–6.0 dan TDS > 640 ppm). Dengan demikian, prototipe ini dinyatakan layak dan valid sebagai alat bantu otomatisasi pada sistem hidroponik.

4.2. Analisis dan Pembahasan

4.2.1. Analisis Implementasi Logika Berbasis *Visual Programming*

Penerapan *visual programming* menggunakan N8N dalam penelitian ini berhasil mengubah kontrol konvensional pada *microcontroller* dengan memisahkan tanggung jawab antara perangkat keras dan logika bisnis. Berdasarkan alur yang tergambar pada *Sequence Diagram* Otomatisasi (Gambar 4.9), terlihat adanya pergeseran beban komputasi di mana *microcontroller* ESP32 tidak lagi dibebani dengan logika pengambilan keputusan yang kompleks, melainkan hanya bertugas sebagai pembaca sensor dan pelaksana perintah. Dengan arsitektur ini, seluruh logika bisnis dan kalkulasi kontrol terpusat pada *workflow* di server, sehingga kinerja perangkat keras menjadi lebih ringan dan fokus pada stabilitas pengambilan data serta pengaktifan komponen fisik.

Proses pengambilan keputusan sepenuhnya ditangani oleh server N8N yang aktif secara periodik melalui *trigger* waktu setiap 30 menit. Saat siklus ini berjalan, server meminta data lingkungan terbaru melalui protokol MQTT, membandingkannya dengan nilai ambang batas di *database*, menghitung durasi pompa, dan mengirimkan instruksi eksekusi balik ke *Device IoT*. Perubahan aturan tanam, seperti penyesuaian kebutuhan nutrisi dan pH saat pergantian fase tanaman, dapat dilakukan secara praktis melalui antarmuka aplikasi *mobile*. Nilai ambang batas baru yang ditambahkan pengguna via aplikasi akan secara otomatis memperbarui tabel *limit_values* di *database*, yang kemudian langsung digunakan oleh logika N8N tanpa perlu melakukan modifikasi manual pada *node* diagram ataupun proses meng-*upload* ulang kode program pada *microcontroller*.

4.2.2. Analisis Struktur Perangkat Lunak

Dari sisi rekayasa perangkat lunak, struktur kode pada *Device IoT* dibangun menggunakan pendekatan Berorientasi Objek untuk memastikan kode yang

modular dan mudah dikelola. Sebagaimana direpresentasikan dalam *Class Diagram* (Gambar 4.15), sistem menerapkan abstraksi perangkat keras melalui kelas-kelas seperti *MainTank*, *MeasuringTank*, *PHCorrector*, dan *TDSCorrector*. Kelas-kelas ini membungkus kerumitan operasional sensor dan aktuator, seperti logika pembacaan sensor kandungan air (*pH*, TDS dan suhu), ultrasonik dan pengendalian akuator (pompa, *valve* dan *mixer*), menjadi metode internal yang rapi. Selain itu, sistem juga menerapkan manajemen *state* menggunakan *Enum StatusState* untuk mencegah konflik operasi. Mekanisme ini memastikan sistem tidak akan menerima perintah kontrol manual ketika sedang sibuk melakukan proses lain, seperti pembacaan sensor atau penambahan larutan koreksi.

4.2.3. Analisis Komunikasi Data dan Basis Data

Keandalan sistem dalam menjaga stabilitas operasional didukung oleh mekanisme komunikasi yang dirancang secara terstruktur untuk menangani proses mekanis yang kompleks. Mengacu pada *Sequence Diagram Membaca Data Lingkungan* (Gambar 4.14), sistem menerapkan metode eksekusi campuran antara proses asinkron (*asynchronous*) dan sekuensial. Pada tahap inisialisasi pembacaan, *Device IoT* menjalankan pengecekan volume cairan pada berbagai tangki (utama dan koreksi) secara paralel di latar belakang. Pendekatan ini memungkinkan sistem untuk mendeteksi kondisi kritis, seperti level air minimum, dan mengirimkan peringatan (*warning*) ke pengguna tanpa harus menghentikan atau memblokir alur utama pengambilan data. Setelah validasi level air selesai, sistem beralih ke proses sekuensial untuk tahap inti pengukuran—mulai dari pengadukan, pengisian tangki pengukuran, pembacaan sensor, hingga pengosongan tangki pengukuran—guna menjamin sampel air yang dibaca homogen sebelum data akhirnya dipublikasikan ke topik MQTT “*environment/data*”.

Selain manajemen alur eksekusi, integritas data menjadi prioritas utama dengan disematkannya mekanisme *fail-safe* pada lapisan komunikasi. Sebagaimana digambarkan dalam alur sistem, perangkat telah mengantisipasi potensi instabilitas jaringan saat pengiriman data. Apabila terjadi kegagalan saat memublikasikan data ke topik utama, perangkat secara otomatis akan mengalihkan pengiriman paket data tersebut ke topik cadangan “*history/pending*”. Server N8N kemudian bertugas memproses pesan dari topik darurat ini dan memecahnya kembali untuk disimpan

ke tabel yang sesuai. Mekanisme ini memastikan bahwa tidak ada data historis yang hilang meskipun perangkat mengalami gangguan koneksi sesaat saat siklus berjalan.

Dari sisi penyimpanan, validitas analisis didukung oleh desain basis data relasional yang memfasilitasi pelacakan historis secara menyeluruh. Struktur *Entity Relationship Diagram (ERD)* sistem menempatkan tabel *records* sebagai pusat data parameter lingkungan (*pH*, TDS, Suhu) yang terhubung langsung dengan tabel pendukung penting lainnya. Setiap entri data terhubung dengan tabel *limit_values* untuk menyimpan konteks aturan ambang batas yang berlaku saat data diambil, serta terhubung dengan tabel *pump_histories* yang mencatat aksi koreksi. Relasi antar-tabel ini memungkinkan peneliti melakukan analisis pasca-kejadian untuk memvalidasi apakah durasi dan jenis pompa yang aktif pada *pump_histories* benar-benar memberikan dampak perubahan nilai yang diharapkan pada data *records* berikutnya.

4.2.4. Evaluasi Kinerja Otomatisasi

Berdasarkan data pengujian selama 72 jam, sistem terbukti mampu menjalankan siklus koreksi otomatis dengan ritme yang konsisten. Waktu respons sistem untuk melakukan satu siklus koreksi—dihitung dari deteksi kondisi tidak ideal hingga penyelesaian tindakan koreksi—sangat dipengaruhi oleh proses mekanis pada perangkat keras. Setiap siklus koreksi melibatkan tahapan fisik yang meliputi: pengadukan larutan di tangki utama selama 1 menit untuk memastikan homogenitas, pengisian tangki pengukuran sekitar 11 detik, pembacaan sensor, dan diakhiri dengan pengosongan tangki ukur yang memakan waktu kurang lebih 2 menit. Total durasi proses mekanis ini (± 3 menit) menjelaskan interval waktu antar-data yang terekam pada *database* saat melakukan koreksi berturut-turut.

Dari sisi akurasi data, evaluasi dilakukan terhadap 17 titik data validasi. Hasil perhitungan menunjukkan total selisih absolut *pH* sebesar 12,29 dan total selisih TDS sebesar 133 ppm. Berdasarkan data tersebut, diperoleh nilai *Mean Absolute Error (MAE)* *pH* sebesar 0,72 dan *MAE* TDS sebesar 7,82 ppm. Nilai penyimpangan rata-rata yang tergolong rendah ini mengindikasikan bahwa sistem mampu melakukan akuisisi data lingkungan dengan tingkat presisi yang baik.

Meskipun demikian, terlihat adanya selisih yang cukup lebar pada beberapa titik pengukuran, khususnya pada parameter *pH* yang selisihnya melebihi 1,0. Kesenjangan ini disebabkan oleh spesifikasi perangkat keras yang digunakan dalam prototipe ini. Sensor yang digunakan merupakan modul PH-4502C dan modul TDS Meter v1.0 yang tergolong modul kelas ekonomis (*low-cost grade*), bukan sensor berstandar industri (*industrial grade*) seperti DFRobot Analog Industrial pH Sensor Pro maupun sensor TDS RS485 Modbus. Sensor kelas ekonomis cenderung memiliki sensitivitas yang lebih tinggi terhadap intervensi eksternal, seperti *electrical noise* atau fluktuasi suhu. Sejalan dengan akurasi tersebut, mekanisme kontrol yang dirancang terbukti berfungsi efektif. Sistem secara konsisten mampu mengembalikan kondisi air ke rentang aman melalui penyesuaian bertahap, membuktikan bahwa fungsionalitas logika *visual programming* yang dibangun dapat berjalan dengan baik dalam mengotomatisasi pemeliharaan nutrisi tanaman.